

UT-11: INTERFACES GRÁFICAS DE USUARIO

¿Qué vamos a ver?

- Interfaces gráficas de usuario.
 - La API de Java para desarrollo de GUI. AWT y SWING.
- Jerarquía de clases.
- Creación de proyecto de entorno gráfico con NetBeans
- Primeros componentes y sus usos
- Código fuente detrás del diseño
- Eventos.
 - Concepto de evento.
 - Creación de controladores de eventos.
- Más componentes y sus usos

1. INTERFACES GRÁFICAS DE USUARIO

Hasta ahora hemos profundizado en aspectos fundamentales de la programación, hemos estudiado las estructuras básicas, los elementos, clases y objetos. Hemos analizado el lenguaje programación Java de forma que a estas alturas sabemos cómo codificar los componentes de un programa usando cualquiera de ellos.

Hemos configurado proyectos en Netbeans con un conjunto de clases, pero todos ellos en modo consola. Hoy en día, con los ordenadores cada vez más potentes, las aplicaciones son sobre todo gráficas y fáciles de usar.

En esta unidad vamos a aprender a desarrollar entornos gráficos, así como los fundamentos de este tipo de aplicaciones en las que el usuario interactúa directamente sobre los componentes.

1.1. La API de Java para desarrollo de GUI. AWT y SWING

Netbeans no distingue a la hora de generar un proyecto entre proyectos gráficos o de consola. Será el programador el que determine de qué tipo será un proyecto después de haber sido creado, mediante la inserción de un formulario o no y la codificación de su método principal.

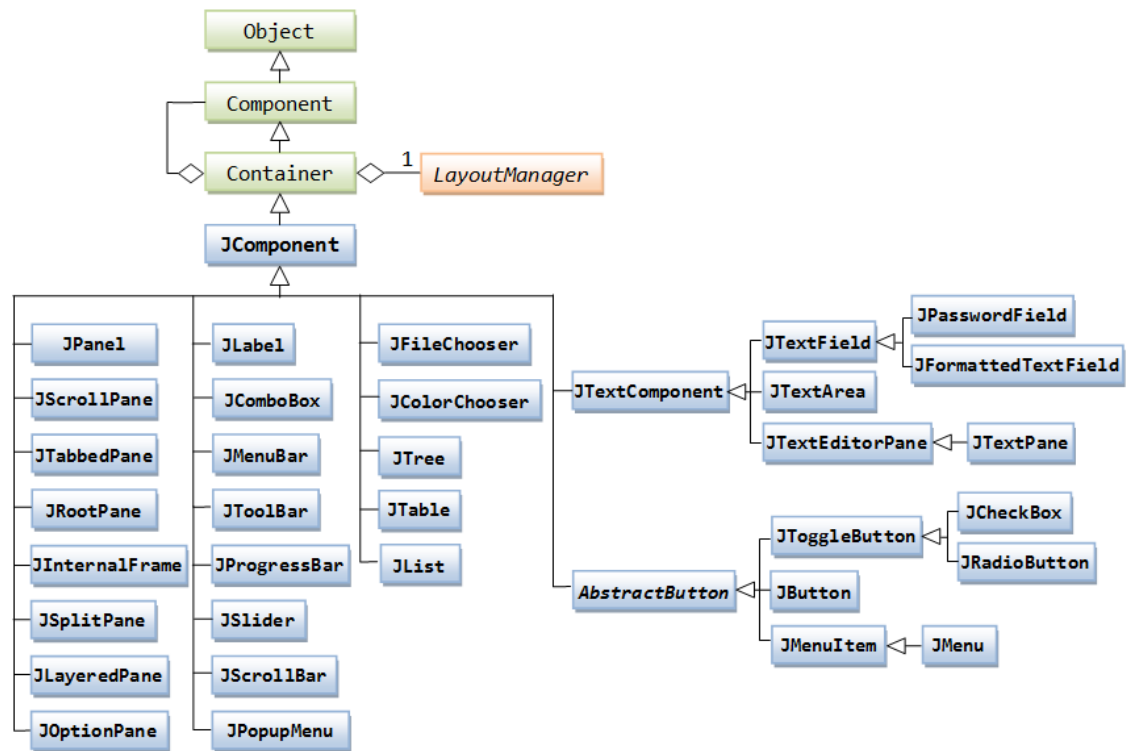
Antes de comenzar a hablar de los controles de Java, es preciso que hablemos de las bibliotecas que usa el lenguaje para el desarrollo de **Interfaces de Usuario Gráficas (GUIs)**. Cuando creemos nuestro proyecto, y observemos el cuadro de herramientas o paleta, veremos diferentes categorías, entre las que se distinguen **AWT y SWING**.

AWT (Abstract Windowing Toolkit) contempla el primer conjunto de clases que referencian componentes gráficos en Java, primer kit de herramientas gráficas. **SWING** apareció como la versión 2.0 y mejorando el aspecto de los controles y su funcionalidad. Hoy se siguen incluyendo ambas bibliotecas ya que en determinados dispositivos o aplicaciones se hacen más eficientes el uso de una u otra.

En nuestro caso nos centraremos en SWING.

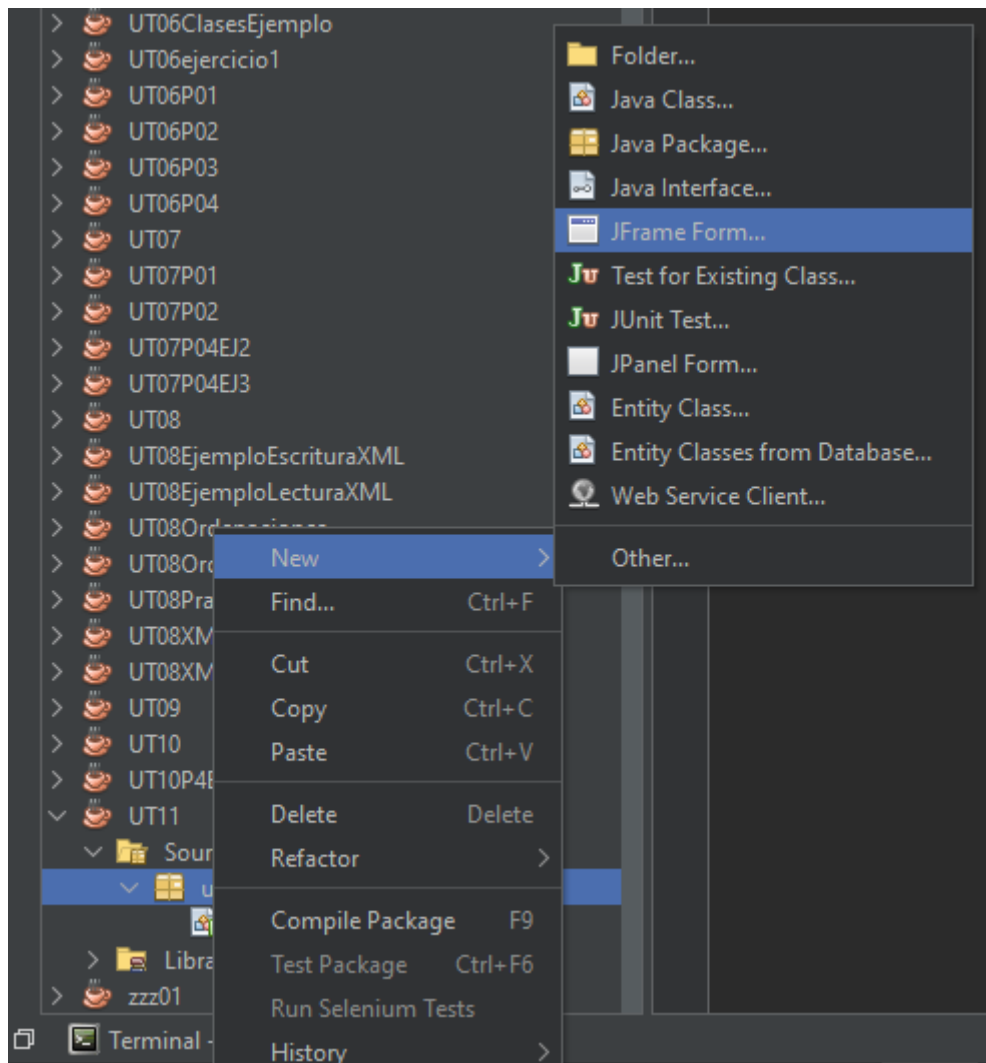
2. JERARQUÍA DE CLASES

A continuación, veamos un esquema que representa las distintas clases de las que disponemos en Swing.

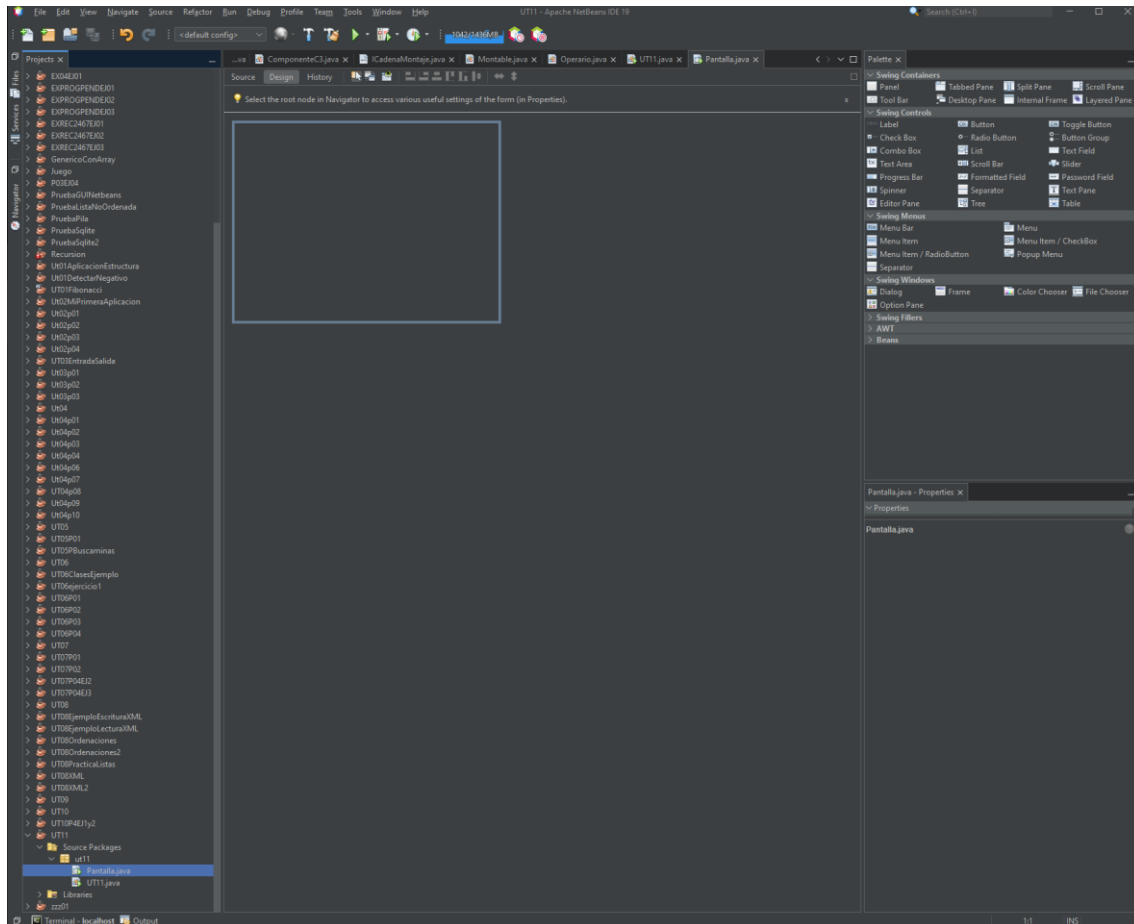


3. CREACIÓN DE PROYECTO DE ENTORNO GRÁFICO CON NETBEANS

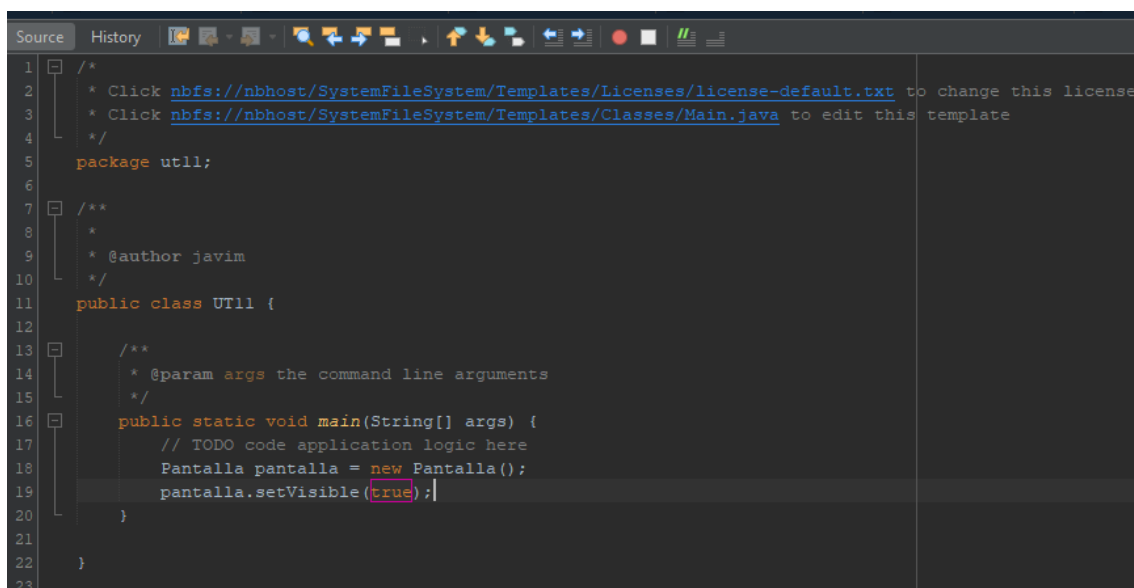
Para crear un proyecto con una interfaz gráfica, debemos hacerlo como si se tratase de un proyecto normal de Java Ant, pero una vez creado le vamos a añadir un JFrame Form, tal y como podemos ver en la imagen.



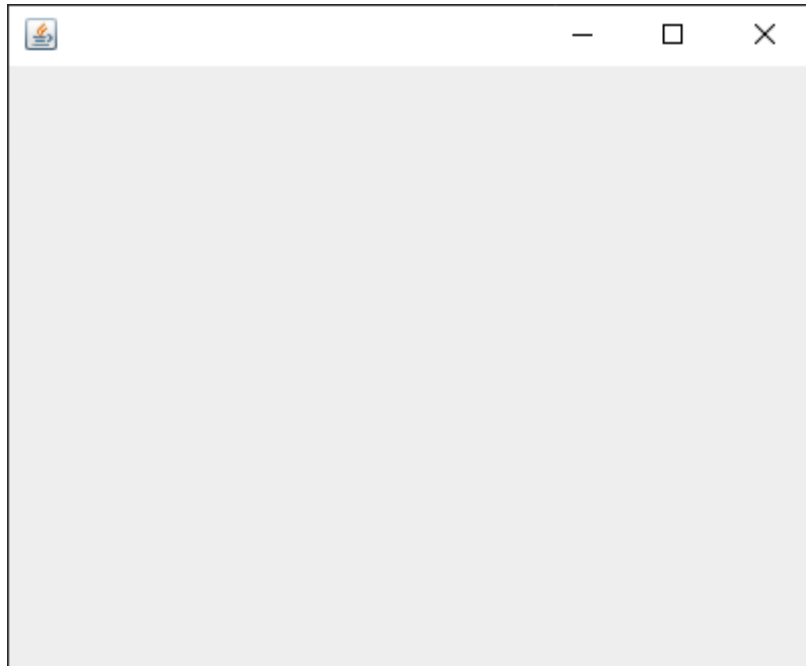
Una vez creada la pantalla, nos aparecerá la ventana de diseño con la paleta de componentes.



Después en el método main, vamos a crear una instancia del formulario y ejecutamos el método setVisible dándole el valor true.

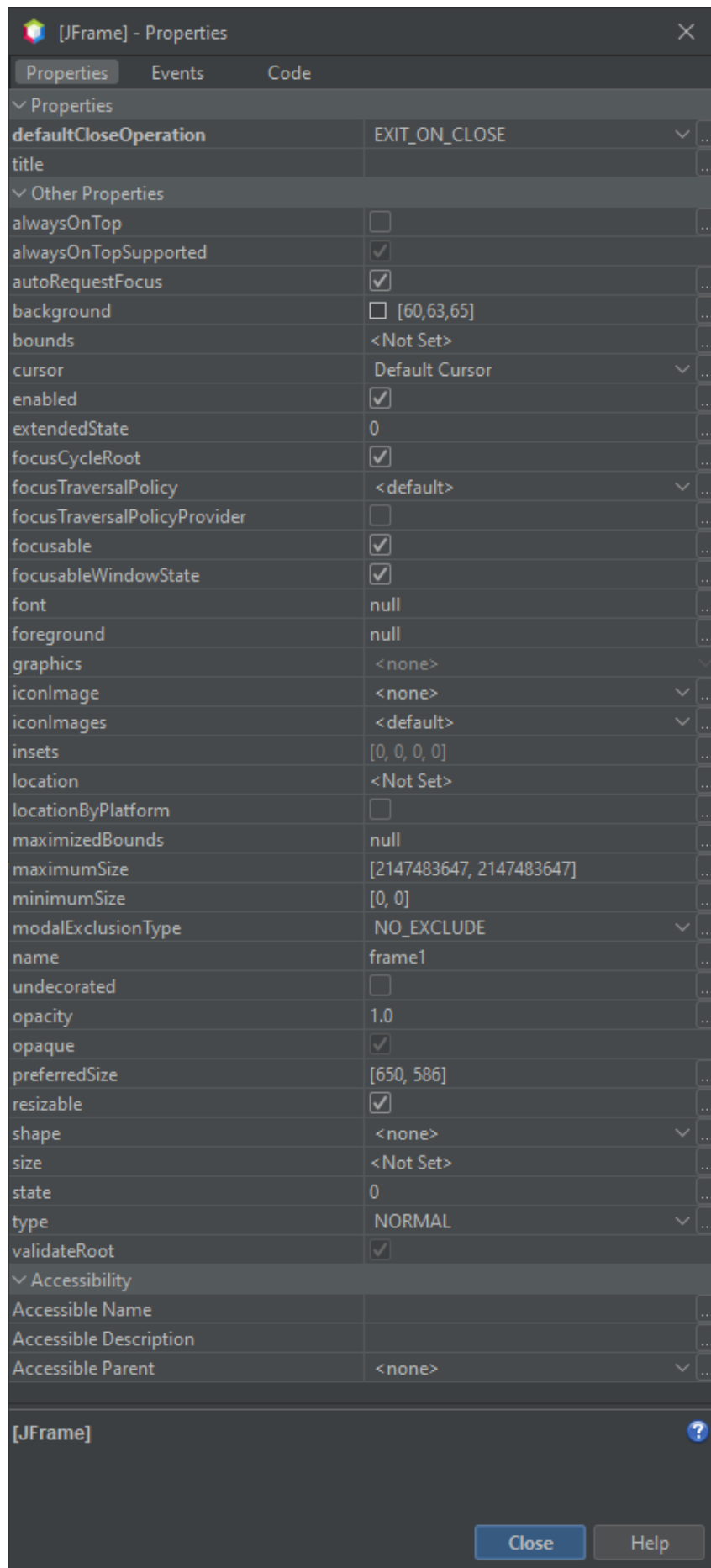


Si ejecutamos el programa nos aparecerá la interfaz gráfica del mismo.

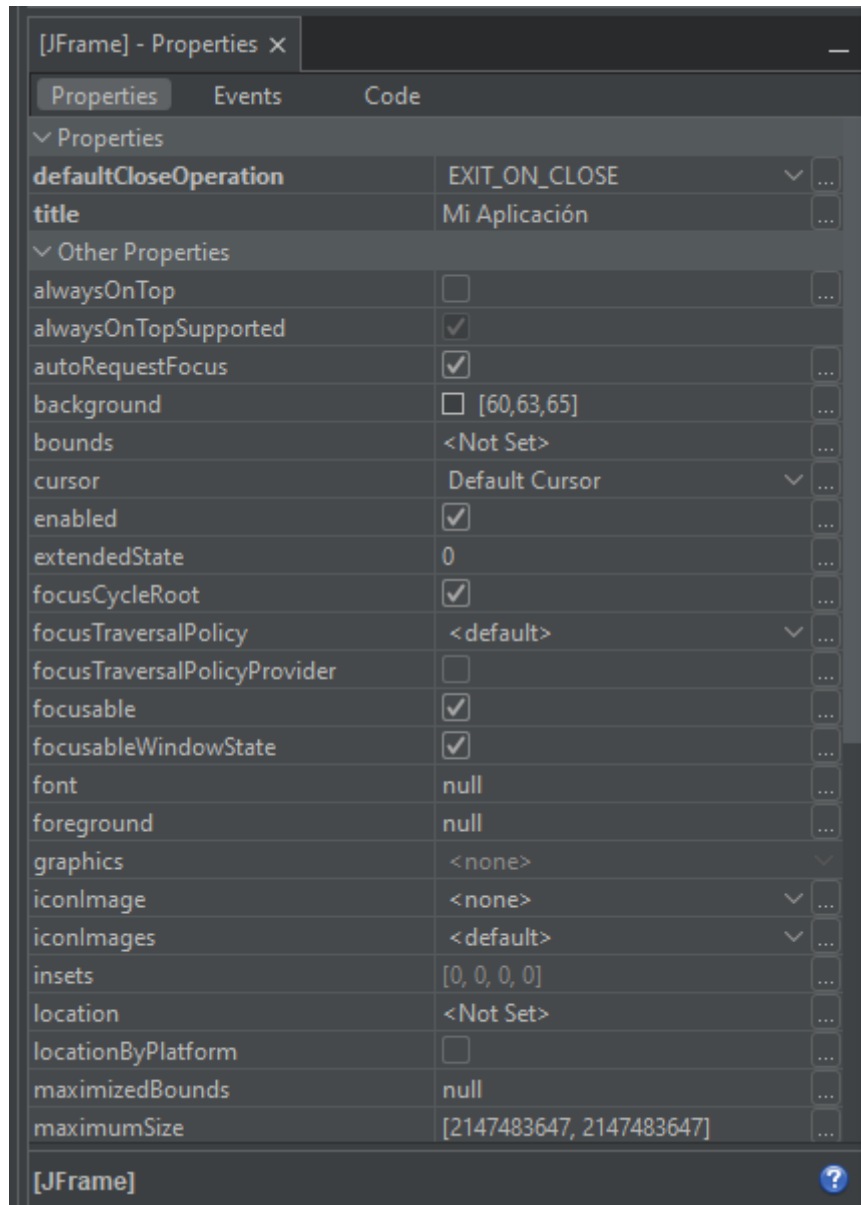


Como aún no hemos añadido ningún componente, ni hemos hecho ninguna modificación solo nos aparece el panel de control de la ventana y un icono por defecto.

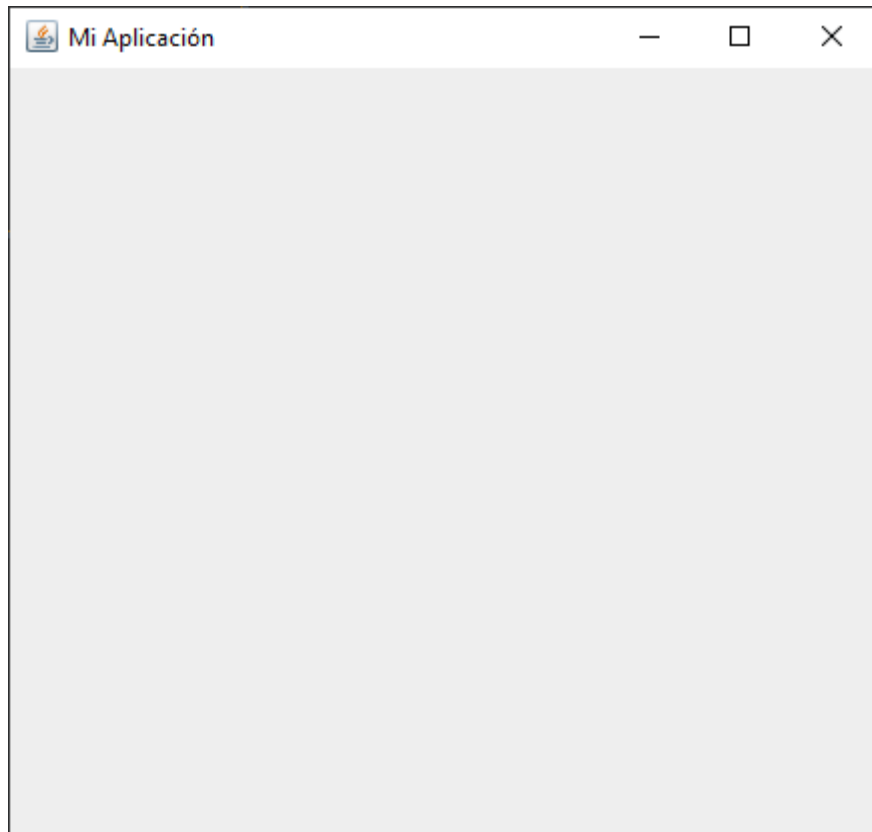
Podemos ponerle un título a nuestra ventana. Si volvemos a la ventana del diseñador y la paleta, nos ponemos sobre el diseño de la ventana y pulsamos botón derecho y seleccionamos la opción “propeties”, nos aparecerá el siguiente cuadro.



Otra opción, es que si pulsamos sobre el componente, debajo de la paleta de componentes nos aparecerá una pestaña properties que es similar a la que hemos mostrado anteriormente:



Si escribimos un texto en el recuadro que está al lado de la propiedad “title”, será el que nos aparezca en la parte superior de nuestra ventana cuando se esté ejecutando.



Con la ventana de “properties” seremos capaces de establecer valores o modificarlos para ajustar nuestra ventana a lo que necesitemos.

Algunas propiedades interesantes son:

- `resizable`: indicaremos si permitimos que la ventana pueda cambiar de tamaño o no.
- `preferredSize`: indicamos el tamaño por defecto que deseamos que tenga la ventana.
- `maximumSize` y `minimumSize`: para indicar el menor y mayor tamaño que queremos que tenga la ventana.

3. PRIMEROS COMPONENTES Y SUS USOS

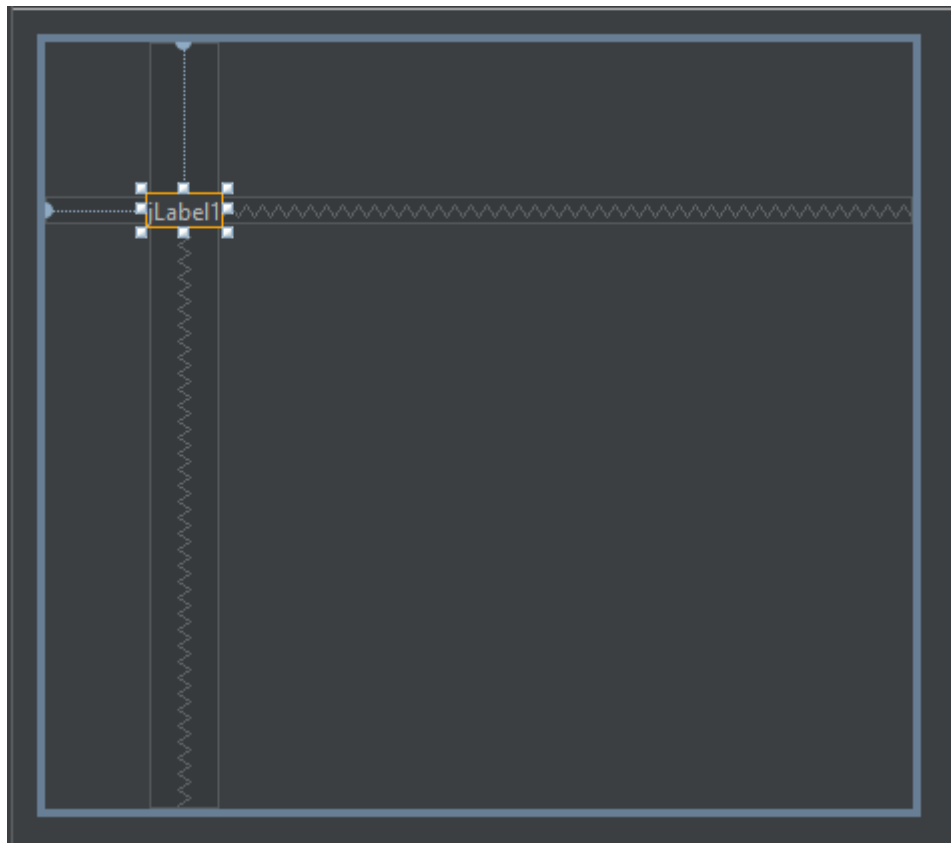
Como ejemplos para aprender a utilizar los primeros componentes nos vamos a centrar en los que están relacionados con textos, es decir:

- Label
- Text Field
- Text Area

Para utilizarlos en nuestra ventana solo tendremos que arrastrarlos de la paleta al diseño de la ventana y colocarlo en la posición que deseemos.

3.1. Label

Este componente lo utilizaremos principalmente para colocar texto en la ventana que el usuario no podrá modificar.



Una vez colocado en el diseño de la ventana podremos moverlo, modificar su tamaño, su texto, ...

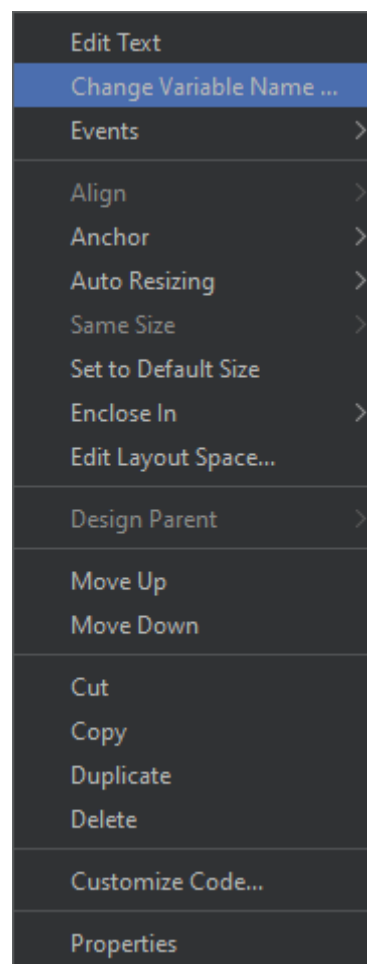
Para moverlo solo tendremos que mantener pulsado el botón izquierdo del ratón sobre él y arrástralo.

Para modificar el tamaño presionaremos y mantendremos el botón izquierdo del ratón sobre uno de los puntos resaltados en los límites del componente.

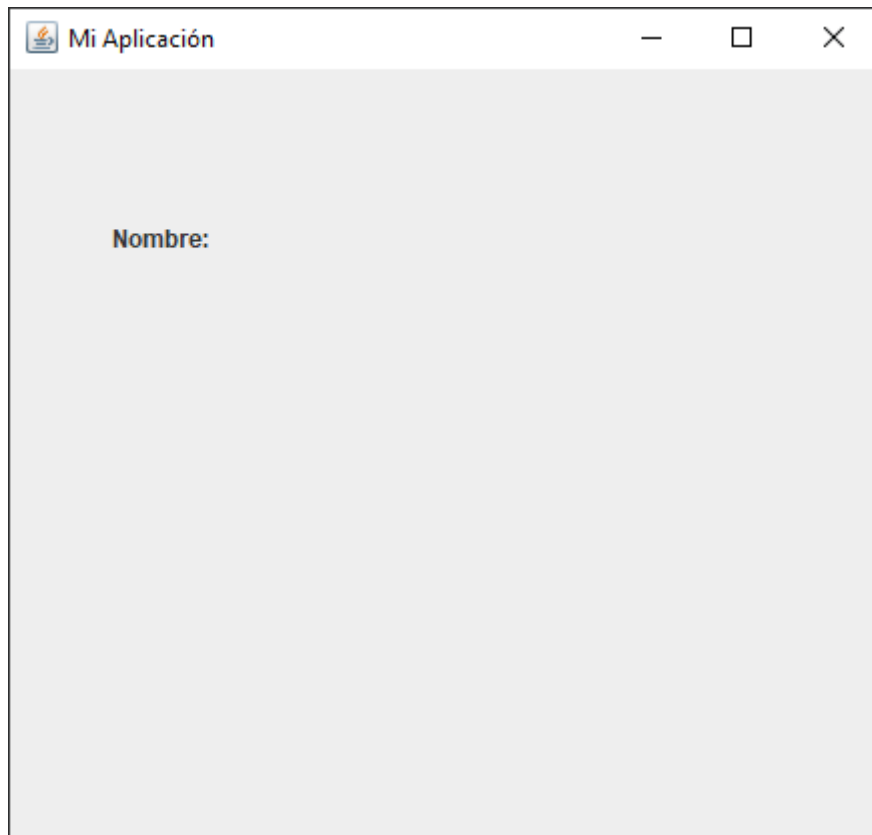
Finalmente, para cambiar su texto y resto de propiedades pulsaremos botón derecho del ratón sobre el mismo y seleccionaremos “properties”, o lo haremos desde la pestaña que nos aparece al seleccionarla debajo de la paleta.

Además de cambiarle el texto, es recomendable cambiar a todos los componentes su nombre de variable, para identificarlo de una mejor forma.

Para hacer este cambio de nombre de variable pulsaremos botón derecho sobre el componente y seleccionaremos “Change variable name”.



Una vez ejecutado, la etiqueta se verá como un texto colocado directamente sobre el fondo de la ventana.



3.2. Text Field

Los text field nos servirán principalmente para que los usuarios nos proporcionen información en forma de texto en una única línea.

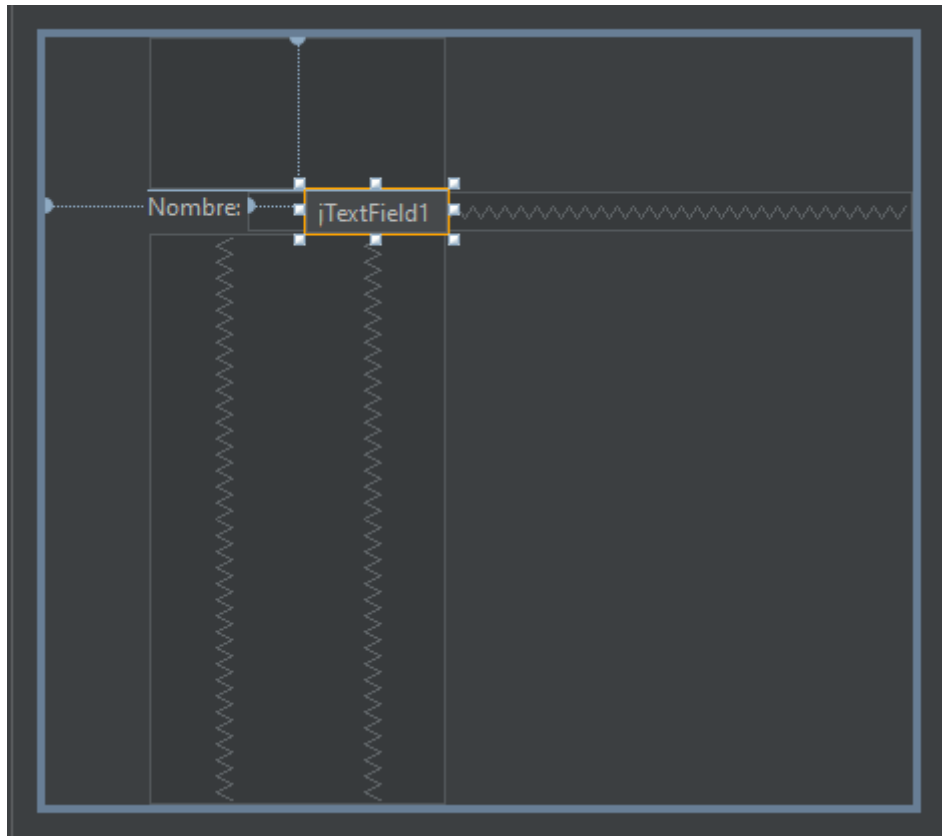
La forma de trabajar con este componente es similar a las etiquetas.

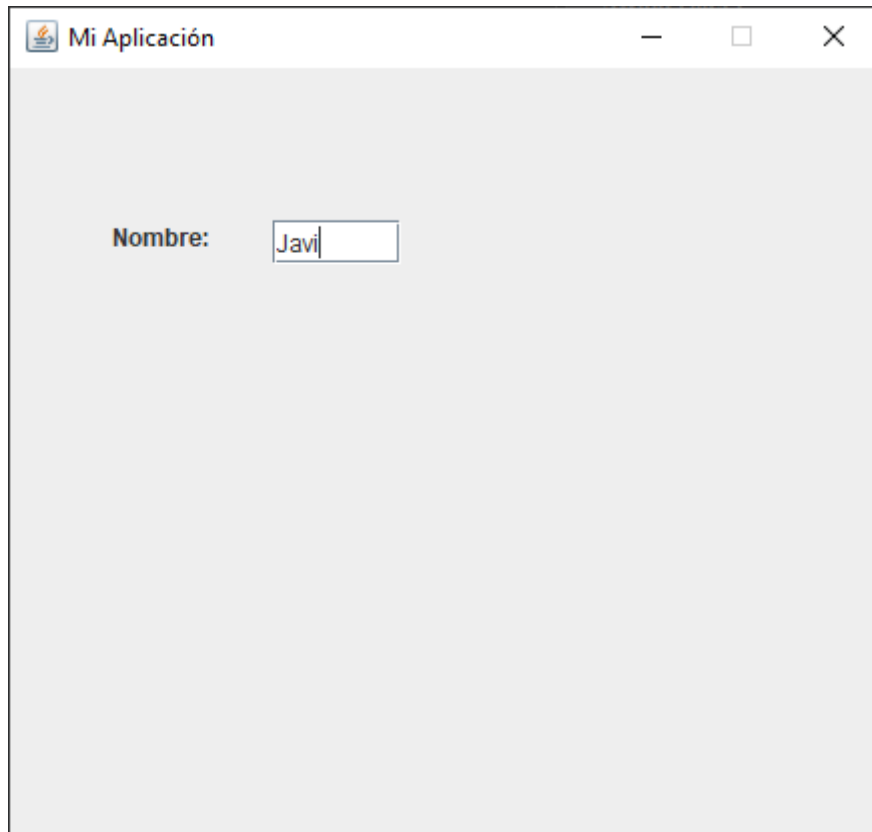
Habitualmente solemos dejar en blanco el texto de estas cajas para que el usuario introduzca su propio texto.

Algunas de las propiedades que más nos puede interesar de los campos de texto son:

- **editable:** lo marcamos o desmarcamos en función de si queremos que se pueda escribir o no en la caja de texto. Pero sí que nos permite hacer otras acciones como seleccionar el texto que contiene.

- enabled: lo marcamos o desmarcamos para habilitar o deshabilitar la caja. Esto solo nos permitirá ver el contenido, pero no podremos realizar ningún tipo de acción sobre ella, ni escribir, ni seleccionar código, ...
- preferred size: para proporcionarle un tamaño por defecto.
- minimum size: para indicarle el tamaño mínimo que deseamos que tenga.



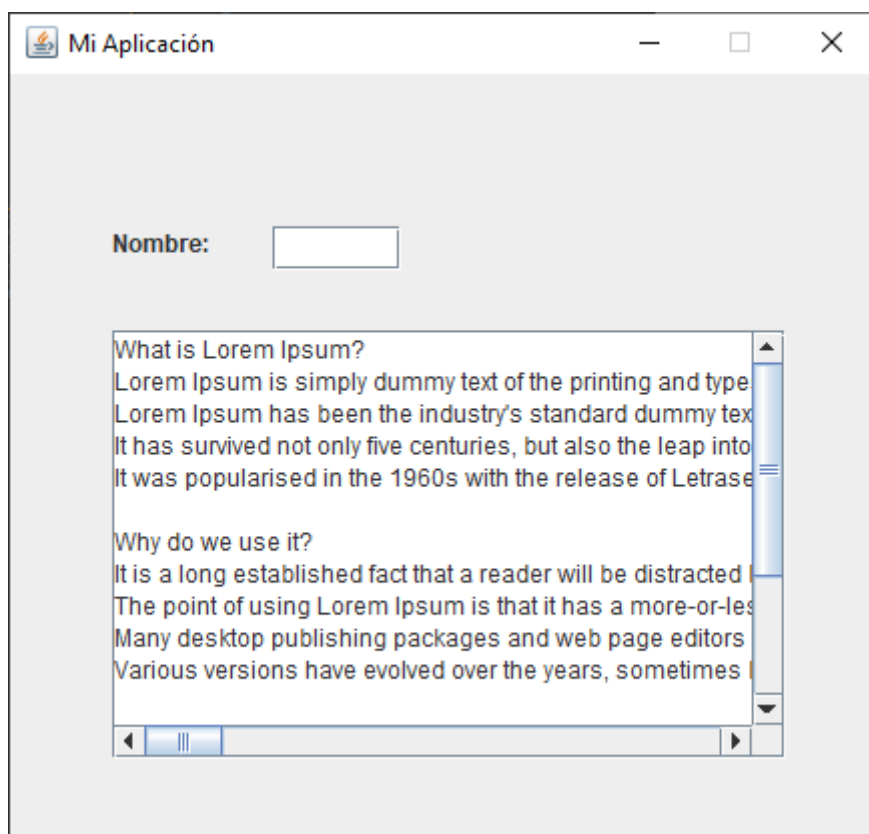
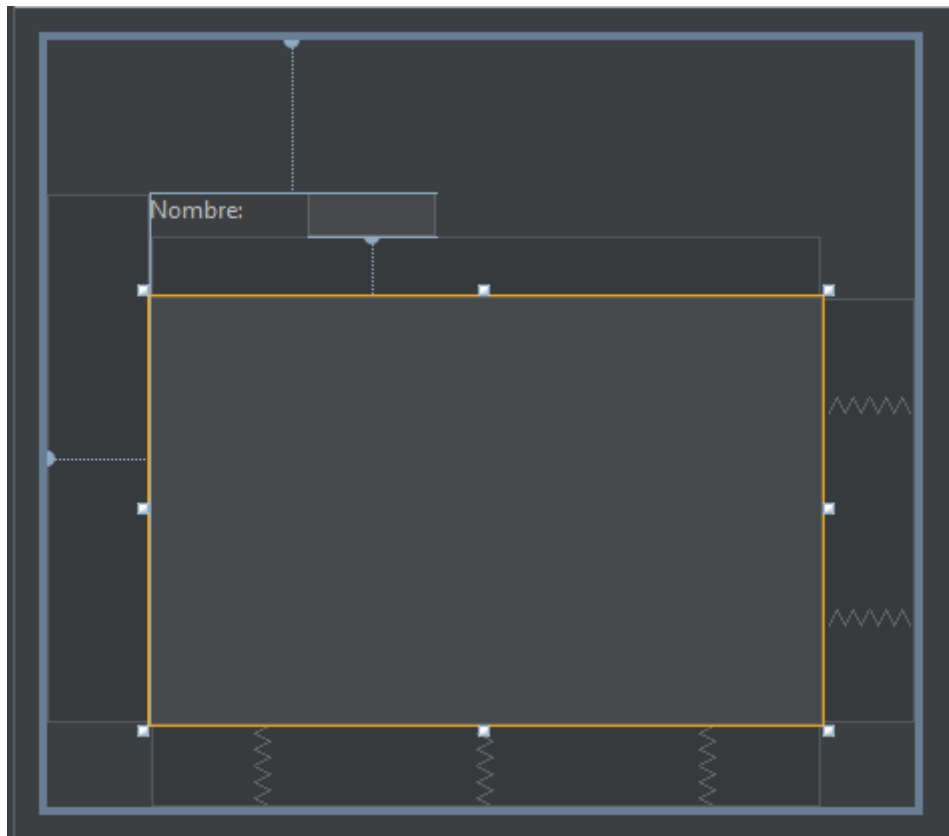


3.3. Text area

Este componente es similar al text field, pero con la diferencia de que en este podemos escribir código en varias líneas.

El text área nos permite introducir textos de gran tamaño, incluso mayor que el ancho y alto del componente, ya que si superamos estos límites aparecerán barras de scroll que nos permitirán.

Si queremos que las líneas se ajusten al ancho del componente debemos poner en true la propiedad **lineWrap**.

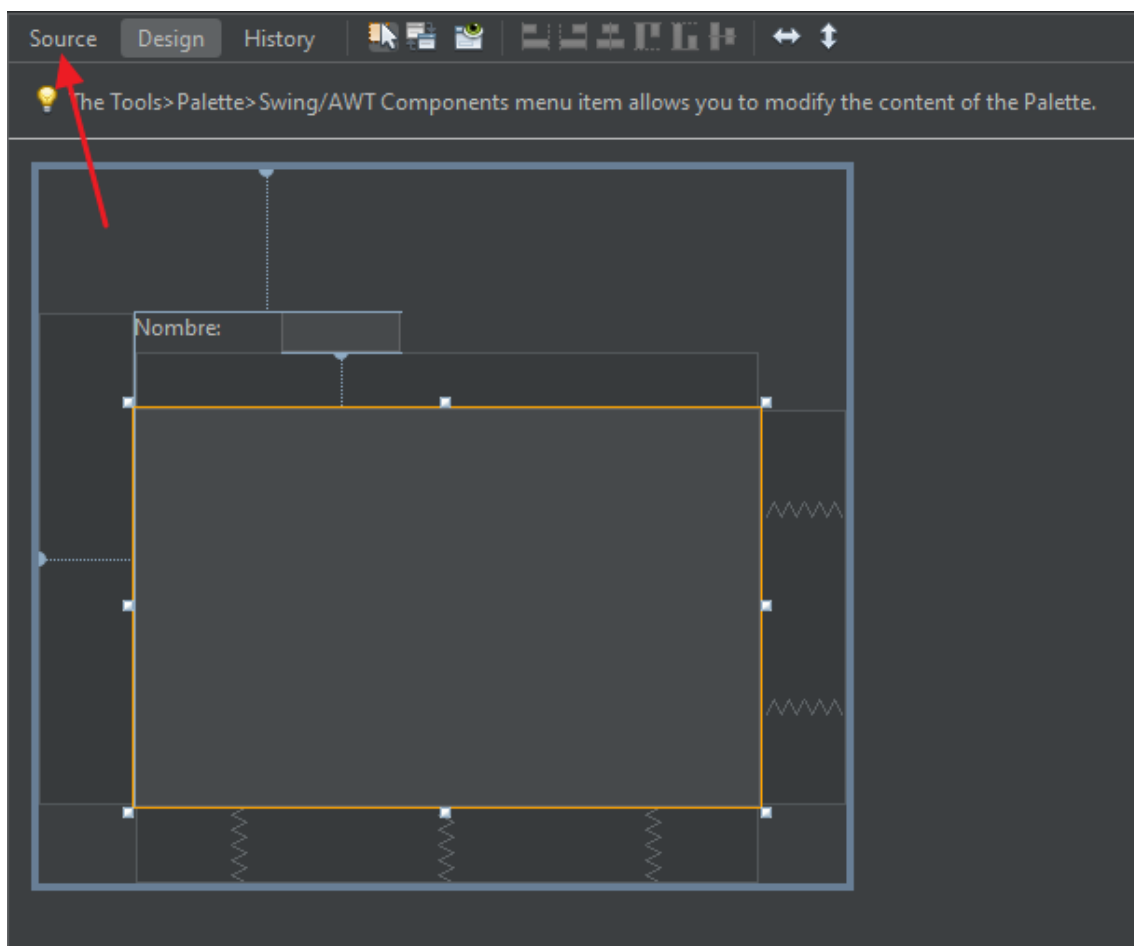


Disponemos de dos formas de trabajar con el texto de un text area:

- Por un lado, la misma que un text field, es decir, el texto en sí, por ejemplo, para obtener el texto podemos utilizar el método `getText()`.
- Y por otro lado, podemos trabajar con el contenido del text area como un documento (Documento), por ejemplo, para obtener propio documento del text area utilizaremos el método `getDocument()`.

4. CÓDIGO FUENTE DETRÁS DEL DISEÑO

En realidad, las ventanas y sus componentes con representaciones gráficas de un código fuente. Podemos ver dicho código si presionamos en la pantalla de diseño la pestaña “Source”.



Esto nos mostrará el código que se ejecuta para que se muestre y funcione la ventana gráfica, y por supuesto podemos modificar dicho código igual que si se tratase de una aplicación de consola.

Veamos el código que se esconde detrás de la ventana que hemos creado hasta el momento.

```
package ut11;

public class Pantalla extends javax.swing.JFrame {

    /**
     * Creates new form Pantalla
     */
    public Pantalla() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize
the form.
     * WARNING: Do NOT modify this code. The content of this method is
always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated
Code">
    private void initComponents() {

        lblNombre = new javax.swing.JLabel();
        txtNombre = new javax.swing.JTextField();
        jScrollPane1 = new javax.swing.JScrollPane();
        txtATexto = new javax.swing.JTextArea();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOS
E);

        setTitle("Mi Aplicación");
        setResizable(false);

        lblNombre.setText("Nombre: ");

        txtNombre.setActionCommand("<Not Set>");
        txtNombre.setMinimumSize(new java.awt.Dimension(128, 22));
        txtNombre.setPreferredSize(new java.awt.Dimension(64, 22));

        txtATexto.setColumns(20);
        txtATexto.setRows(5);
        jScrollPane1.setViewportView(txtATexto);

        javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.
LEADING)
```

```

        .addGroup(layout.createSequentialGroup()
            .addGap(51, 51, 51)
            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.
                Alignment.LEADING)
                .addComponent(jScrollPane1,
                    javax.swing.GroupLayout.PREFERRED_SIZE, 336,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGroup(layout.createSequentialGroup()
                    .addComponent(lblNombre)
                    .addGap(29, 29, 29)
                    .addComponent(txtNombre,
                        javax.swing.GroupLayout.PREFERRED_SIZE,
                        javax.swing.GroupLayout.DEFAULT_SIZE,
                        javax.swing.GroupLayout.PREFERRED_SIZE)))
                .addContainerGap(46, Short.MAX_VALUE))
        );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.
                LEADING)
                .addGroup(layout.createSequentialGroup()
                    .addGap(76, 76, 76)
                    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.
                        Alignment.LEADING)
                        .addComponent(txtNombre,
                            javax.swing.GroupLayout.PREFERRED_SIZE,
                            javax.swing.GroupLayout.DEFAULT_SIZE,
                            javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(lblNombre))
                    .addGap(30, 30, 30)
                    .addComponent(jScrollPane1,
                        javax.swing.GroupLayout.PREFERRED_SIZE, 214,
                        javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addContainerGap(41, Short.MAX_VALUE))
                );

        pack();
    }// </editor-fold>

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel
        setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay
        with the default look and feel.
        * For details see
        http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html

```

```

        */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
                javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {
            java.util.logging.Logger.getLogger(Pantalla.class.getName()).
                log(java.util.logging.Level.SEVERE, null, ex);
        } catch (InstantiationException ex) {
            java.util.logging.Logger.getLogger(Pantalla.class.getName()).
                log(java.util.logging.Level.SEVERE, null, ex);
        } catch (IllegalAccessException ex) {
            java.util.logging.Logger.getLogger(Pantalla.class.getName()).
                log(java.util.logging.Level.SEVERE, null, ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {
            java.util.logging.Logger.getLogger(Pantalla.class.getName()).
                log(java.util.logging.Level.SEVERE, null, ex);
        }
        //</editor-fold>
        //</editor-fold>

        /* Create and display the form */
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new Pantalla().setVisible(true);
            }
        });
    }

    // Variables declaration - do not modify
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JLabel lblNombre;
    private javax.swing.JTextArea txtATexto;
    private javax.swing.JTextField txtNombre;
    // End of variables declaration
}

```

Como puedes ver, en el constructor de la clase Pantalla es donde inicializamos todos los componentes. A su vez, en esa inicialización es donde se van a definir todas las propiedades que nosotros hemos modificado en diseño.

También tenemos, en la parte inferior, la declaración de todas las variables correspondientes a los componentes, ya que en realidad son variables de una clase determinada.

Y, por último, tenemos un método main en el que se arranca la ventana.

Como ya hemos comentado, nosotros podremos modificar, de una forma controlada, este código, y con ello podremos modificar las propiedades de los componentes durante la ejecución, realizar distintas funcionalidades con nuevos métodos que introduzcamos... y, como veremos más adelante, controlar el comportamiento de los distintos componentes.

5. EVENTOS

5.1. Concepto de evento

Todos los programas desarrollados hasta el momento siguen una misma pauta, presentan un planteamiento secuencial, de forma que comienzan en un punto, van bifurcándose, pero a fin de cuentas somos nosotros, los que desarrollamos el software, los que creamos desde el principio hasta el final el camino que debe seguir la ejecución del programa.

Cuando desarrollamos programas con entornos gráficos esto cambia, ya que no existe una secuencia en las instrucciones, o al menos en las instrucciones que conforman el programa como un todo. En un programa con entorno gráfico se codifican cada parte significativa del entorno, de forma que cuando el usuario interactúe con esta realice alguna acción.

Así, los programas diseñados con entornos gráficos se dicen que son programas **orientados a eventos**, ya que el programa irá respondiendo a los diferentes eventos externos (si hizo click en un botón, se pulsó el botón de cerrar ventana, etc.)

A la hora de crear aplicaciones gráficas estas serán ejecutadas sobre un sistema operativo concreto y partimos de que conocemos el entorno, los componentes que suele utilizar y los eventos o sucesos a los que responde. Cuando creamos un programa gráfico para Windows sabemos que estará compuesto de ventanas,

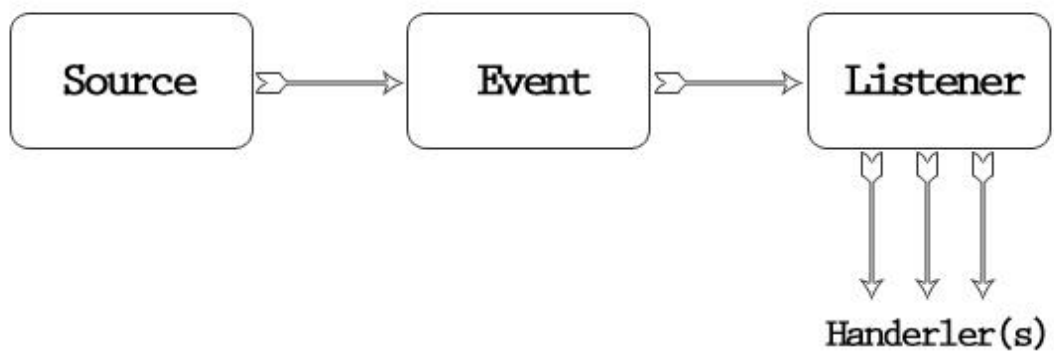
y que estas ventanas tendrán botones, cuadros de texto, etc., que reaccionarán a clicks de ratón, pulsaciones de teclas, etc.

La forma en la que trabajaremos consistirá en manejar eventos y escuchadores (listeners) de eventos.

Los eventos son señales que se lanzarán cada vez que ocurre algo significativo en el programa. Por ejemplo, si pulsamos con el ratón en un botón, se lanzará una señal que informará de ello al sistema.



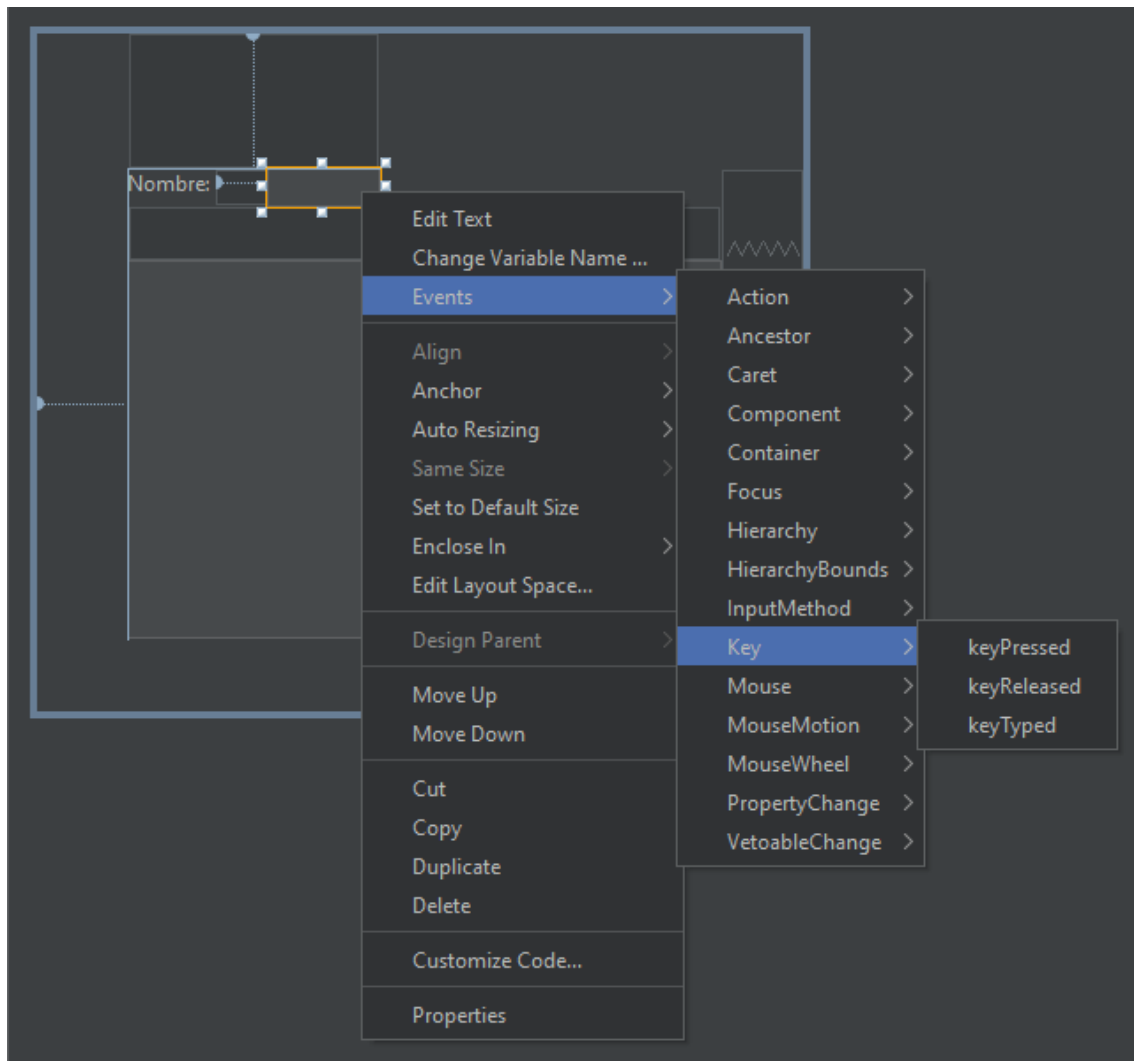
Los listeners (o escuchadores) son métodos que se encuentran a la espera de capturar una señal determinada, y llevarán a cabo una la tarea específica cada vez que esta señal sea detectada.



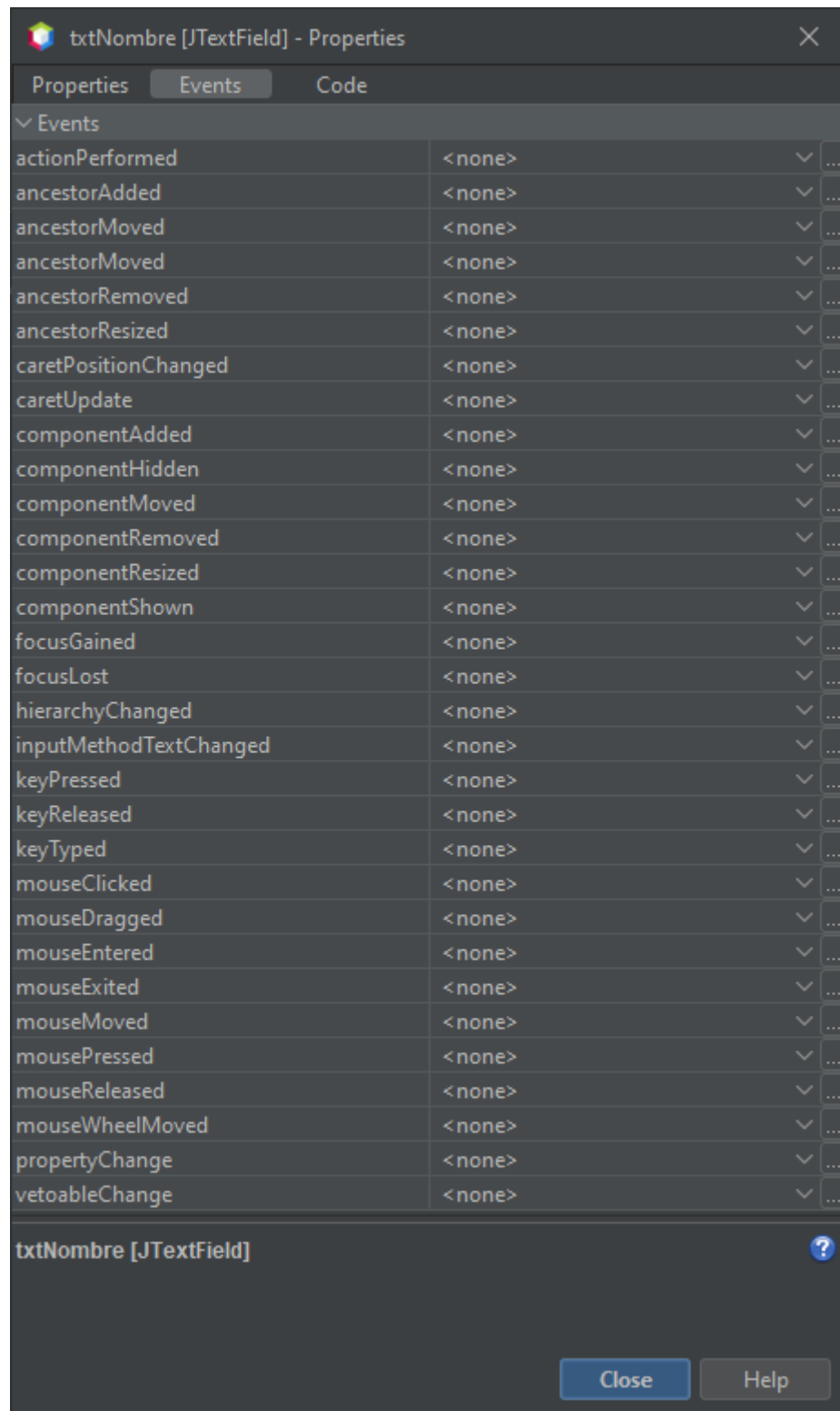
Event-Driving Programming Model

5.2. Creación de controladores de eventos

Para crear un escuchador de un suceso determinado en uno de los componentes en la ventana de diseño, podemos pulsar botón derecho del ratón sobre el componente, ponernos sobre la opción “Events” y se desplegarán las siguientes opciones, de tal forma que solo tendremos que pulsar sobre la que queremos utilizar.



Las otras posibilidades son acceder a Properties, ya sea con botón derecho del ratón, o con la pestaña de eventos que se encuentra debajo de la paleta.



Veamos un ejemplo:

Si queremos que cada vez que pulse una tecla en text field que había creado se cambie el color del texto, primero seleccionaría el evento `keyReleased`. Esto me crearía el siguiente código en Java:

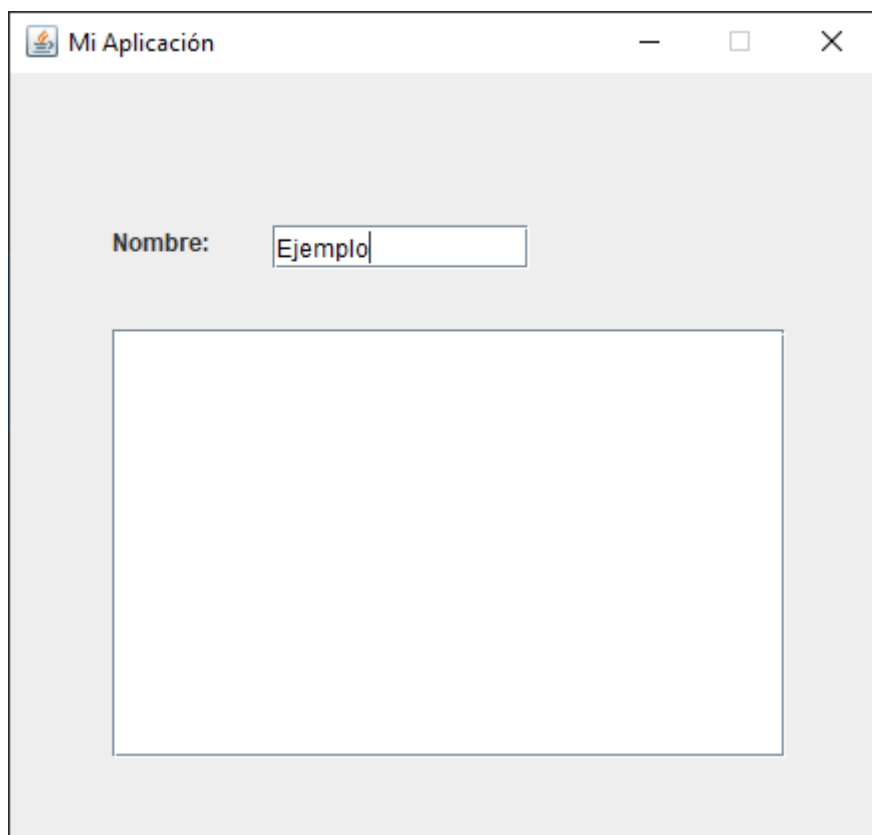
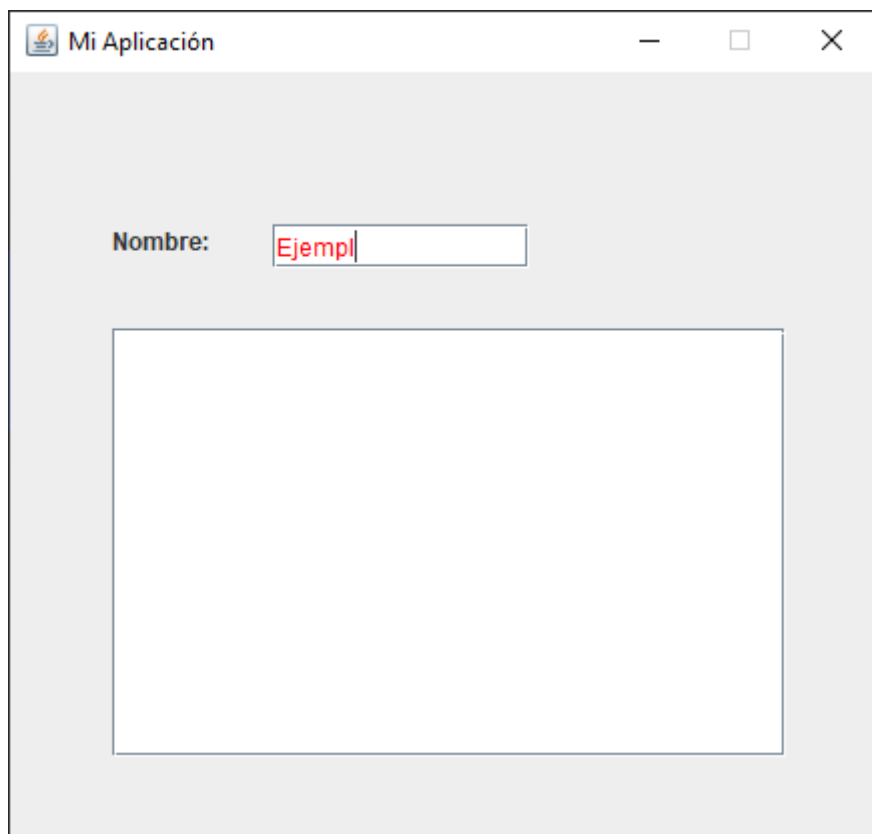
```
private void txtNombreKeyReleased(java.awt.event.KeyEvent evt)
{
    // TODO add your handling code here:
}
```

Si te fijas, el nombre del método consiste en el nombre de variable que le hemos dado al text field (`txtNombre`) + el tipo de evento que queremos capturar (`KeyRelease`). Después aparece el parámetro “`evt`”, que como podemos deducir se trata de un objeto de la clase de eventos de teclas.

Una vez creado el método, nosotros tendremos que rellenar su contenido para indicar qué debe hacerse cada vez que se pulse una tecla en ese text field.

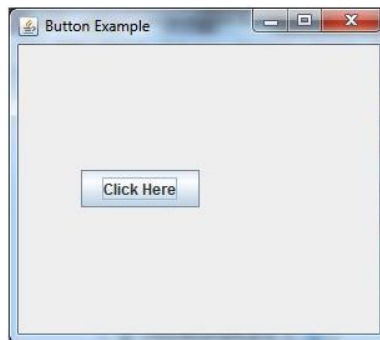
El siguiente sería un posible código que va cambiando el color del texto a cada pulsación que vamos dando, entre negro y rojo.

```
private void txtNombreKeyReleased(java.awt.event.KeyEvent evt)
{
    if (txtNombre.setForeground().equals(Color.BLACK))
        txtNombre.setForeground(new Color(255,0,0));
    else
        txtNombre.setForeground(Color.BLACK);
}
```



6. MÁS COMPONENTES Y SUS USOS

6.1. Botones

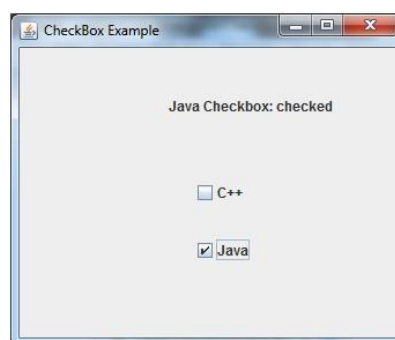


Los botones son unos de los componentes con los que más interactúan los usuarios. Nos permiten realizar una acción determinada cada vez los pulsamos.

Los botones tienen bastantes propiedades comunes a otros componentes que ya hemos visto como `text`, `enabled`, `minimumSize`, `preferredSize`, ...

Además, podemos hacer uso de distintos manejadores de eventos, siendo el más habitual `clicked`, pero también podremos gestionar cuando el ratón se ponga encima de del botón, cuando lo abandone, etc...

6.2. Check box

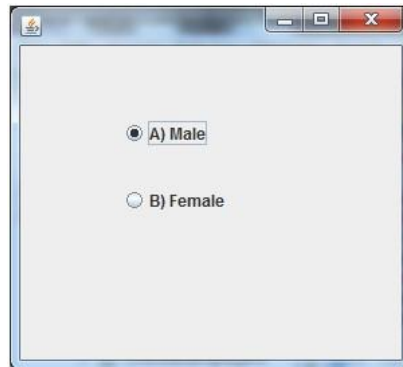


Podemos tomar los check box como un componente similar a un valor booleano, que únicamente puede tomar dos valores, marcado o no marcado. Esto lo comprobaremos por código mediante el método `isSelected` y le asignaremos valor por medio del método `setSelected(boolean)`.

Cada uno de los check box que hay en una misma ventana son independientes.

Con respecto a los eventos, tendremos disponibles la misma variedad que el resto de componentes.

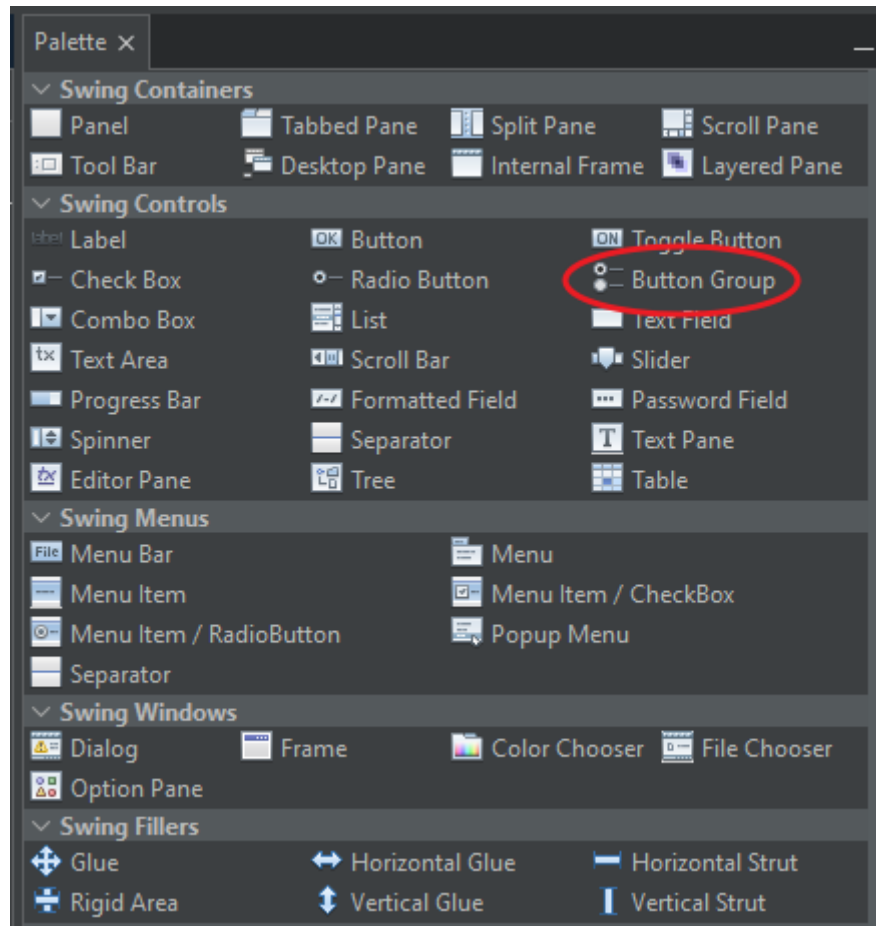
6.3. Radio button y button group



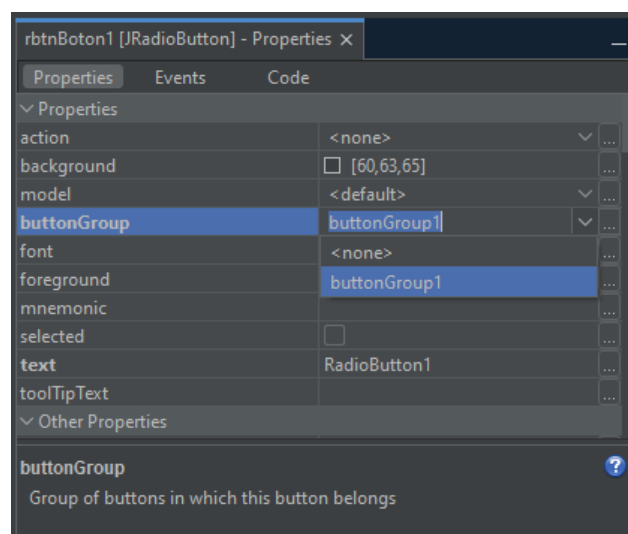
Los radio button son similares a los check box desde el punto de vista que también solo cuentan con dos valores, marcado o desmarcado. La diferencia existente con los check box es que, además de utilizarlos de forma independiente, también podemos asociarlos de tal forma que si tenemos varios radio buttons, cuando uno está marcado, el resto se mantendrá desmarcado. Es decir, dentro del mismo grupo, solo se podrá tener marcado un radio button.

Esto vamos a poder realizarlo gracias a asociar los radio button a un mismo button group, que será el encargado de que solo se encuentre marcado uno de los radio buttons que están asociados al mismo.

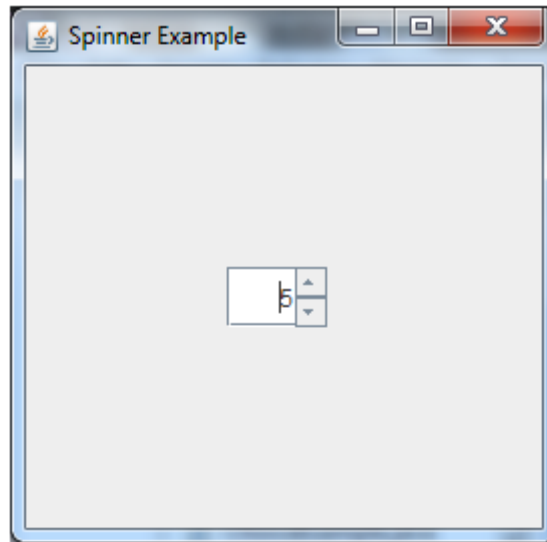
Los primero que tendremos que hacer es pinchar en button group de la paleta y arrastrarlo a la ventana de diseño. El button group, desaparecerá, pero seguirá ahí.



Para asociar un radio button a un button group, vamos a sus propiedades y tenemos una denominada `buttonGroup`, que tiene un desplegable. Ahí seleccionaremos el button group al que queremos asociarle.



6.4. Spinner

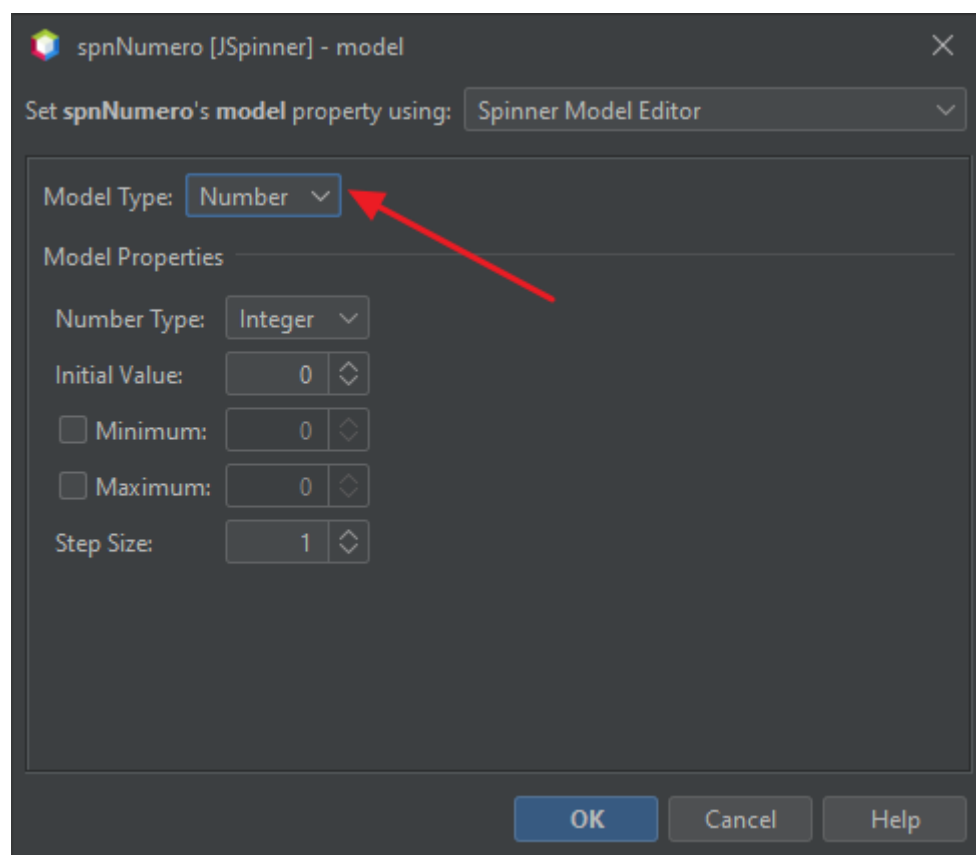
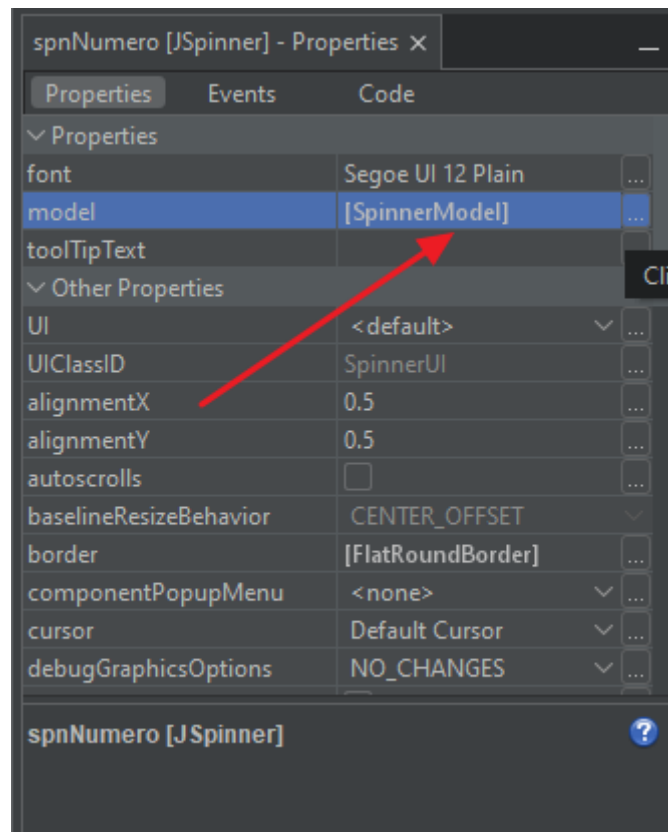


Con el componente Spinner vamos a poder introducir valores numéricos enteros.

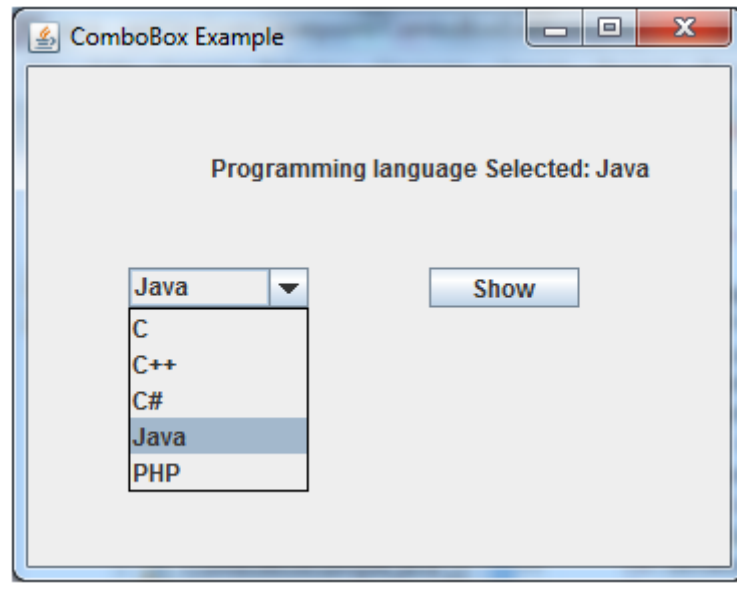
Podremos proporcionar directamente el valor por teclado, o podremos indicarlo por medio de las flechas arriba y abajo que incluye.

Para recoger su valor utilizaremos el método `getValue()` y para proporcionarle un valor por código utilizaremos el método `setValue(int numero)`.

En el caso de que necesitemos limitar los valores que puede tomar el Spinner iremos a sus propiedades y seleccionaremos la propiedad `model`. Esto nos abrirá un cuadro de diálogo, donde seleccionaremos la opción `number`, y aparecerán los cuadros donde podremos poner el tipo de número, el valor inicial, el valor máximo, el mínimo y los pasos en los que se moverá.



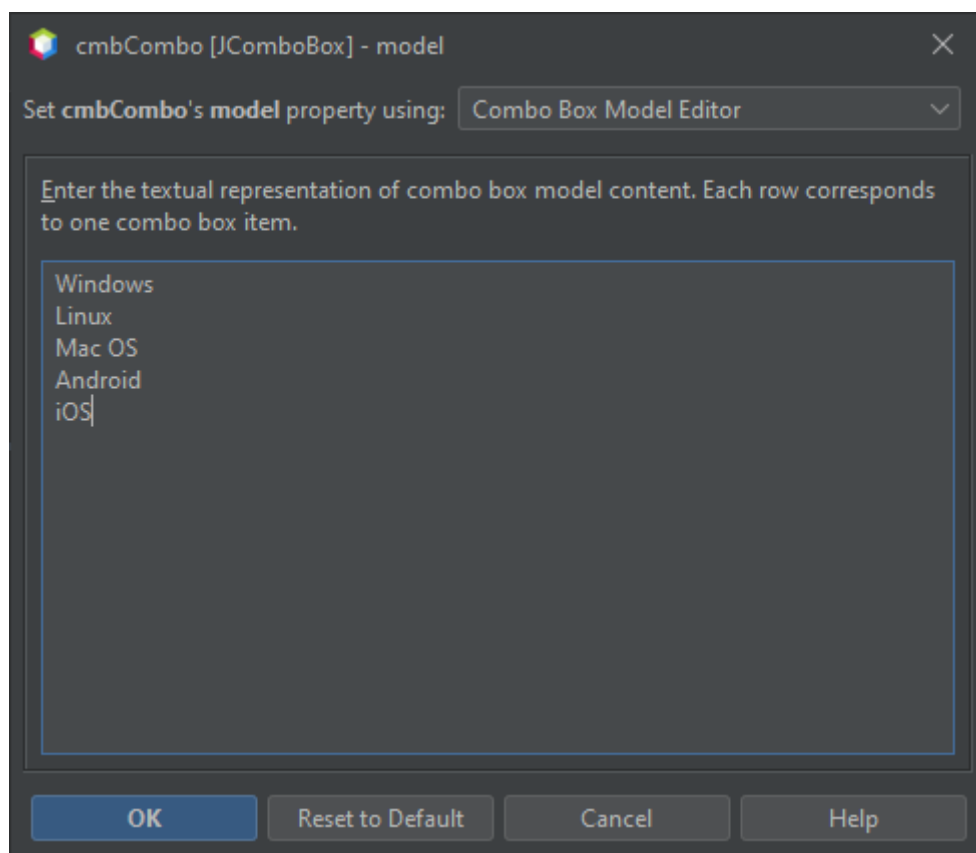
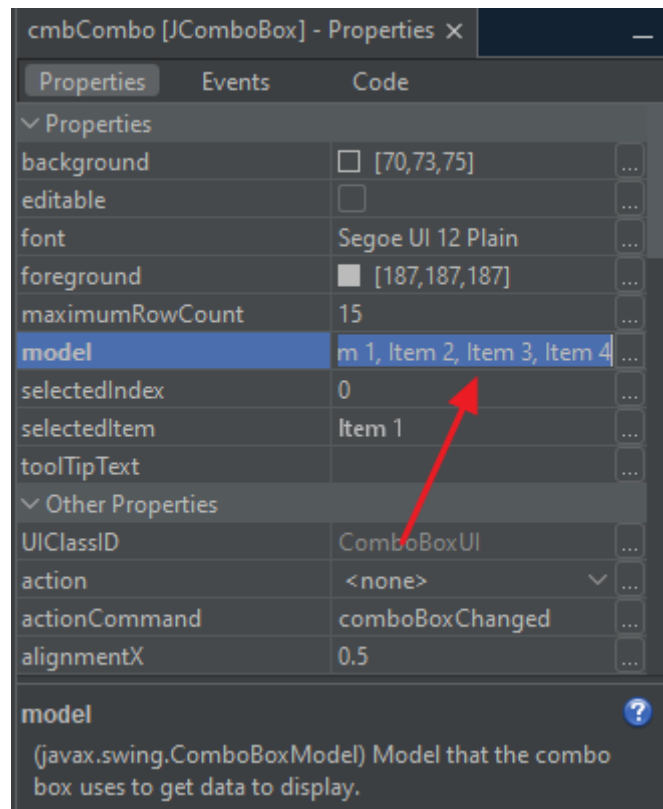
6.5. Combo box



Los combo box nos permitirá elegir una opción en varias que nos muestre.

Para introducir valores al combo box podemos hacerlo por diseño o por código.

Para hacerlo por diseño, como siempre, nos iremos a las propiedades del componente, seleccionaremos model y nos aparecerá una ventana con las distintas opciones puestas por defecto. Ahora las modificaremos para que en cada una de las líneas pongamos las opciones que necesitamos.



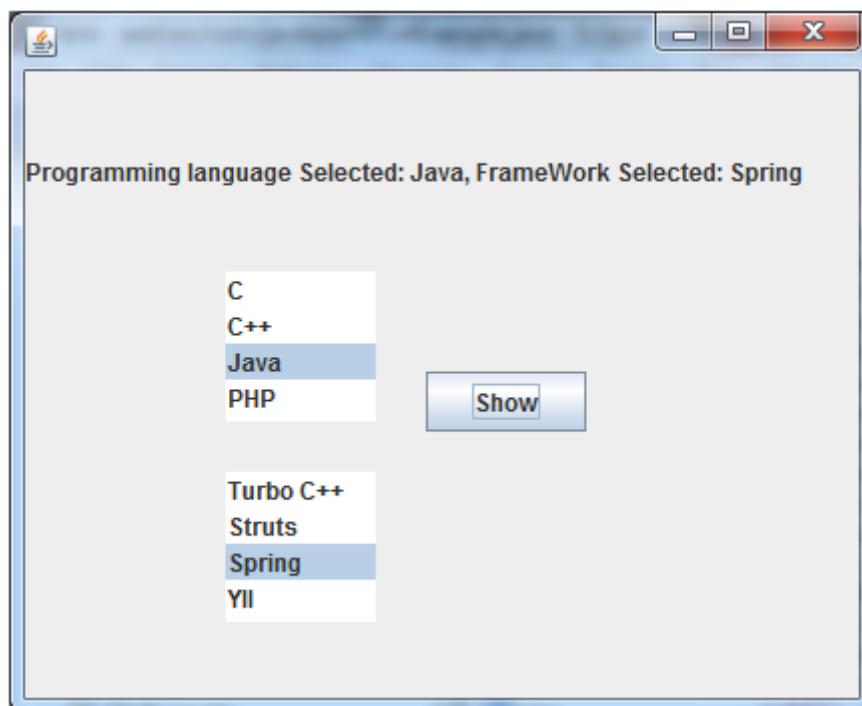
Si lo que necesitamos es proporcionar los valores por código, lo haremos mediante distintos métodos como:

- addItem(String item)
- removeAllItems()
- removeItemAt(int index)

Además, por código podremos realizar distintos métodos interesantes como:

- getItemAt(int index)
- getItemCount()
- getSelectedItem()
- getSelectedIndex()

6.6. List



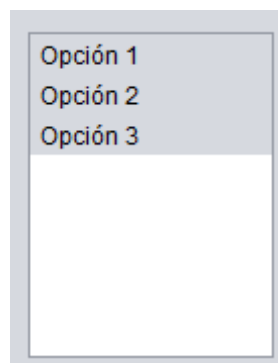
List box es similar a combo box, con la diferencia de que vemos continuamente todos los ítems y, además, podremos configurarlo para que puedan seleccionar un solo ítem a la vez, varios, o un intervalo.

A la hora de introducir los datos, se realiza de la misma forma que los combo box a través de la ventana de diseño.

Pero si queremos hacerlo por código, funciona de una forma distinta, ya que las listas cargan la información de un DefaultListModel. Por ello, los pasos a seguir son:

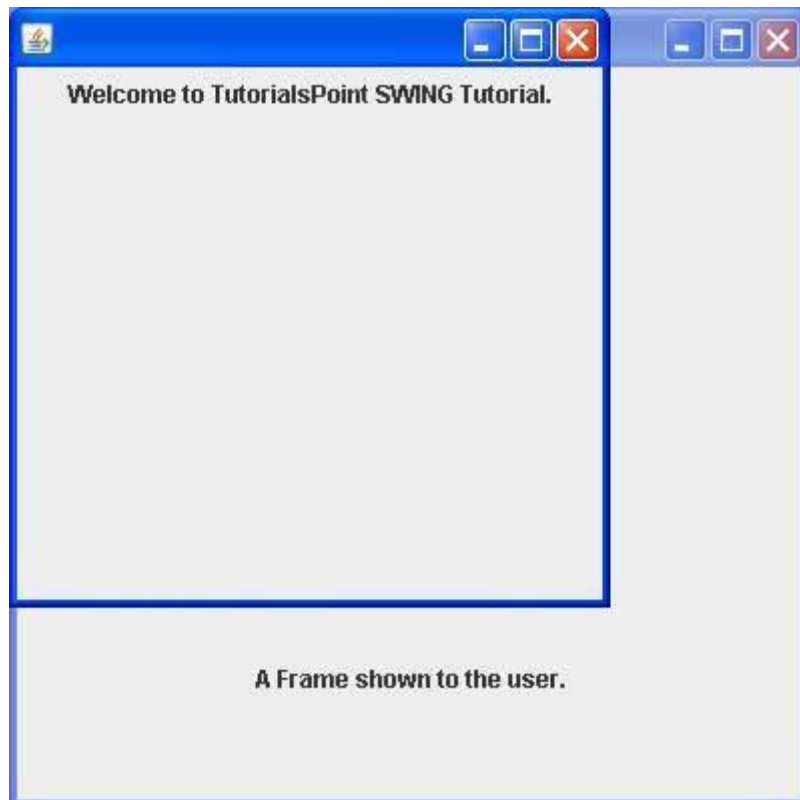
1. Crear el DefaultListModel
2. Añadir los ítems al DefaultListModel (igual que si fuese una lista normal)
3. Vincular el DefaultListModel a la List

```
DefaultListModel modelo = new DefaultListModel();  
modelo.addElement("Opción 1");  
modelo.addElement("Opción 2");  
modelo.addElement("Opción 3");  
lstLista.setModel(modelo);
```



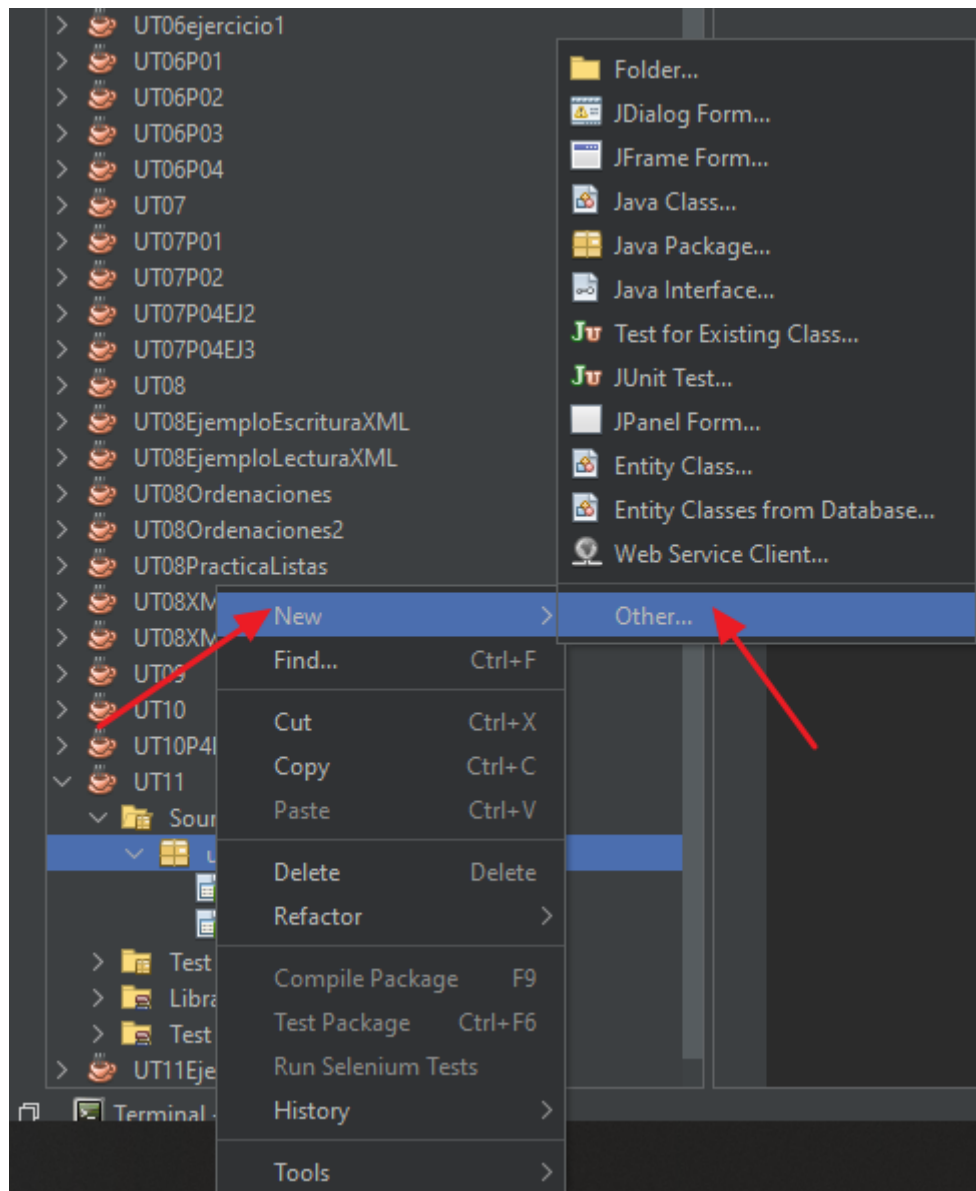
Al igual que el combo box, podremos seleccionar objetos en un determinado índice, eliminarlos, etc...

6.7. Diálogos

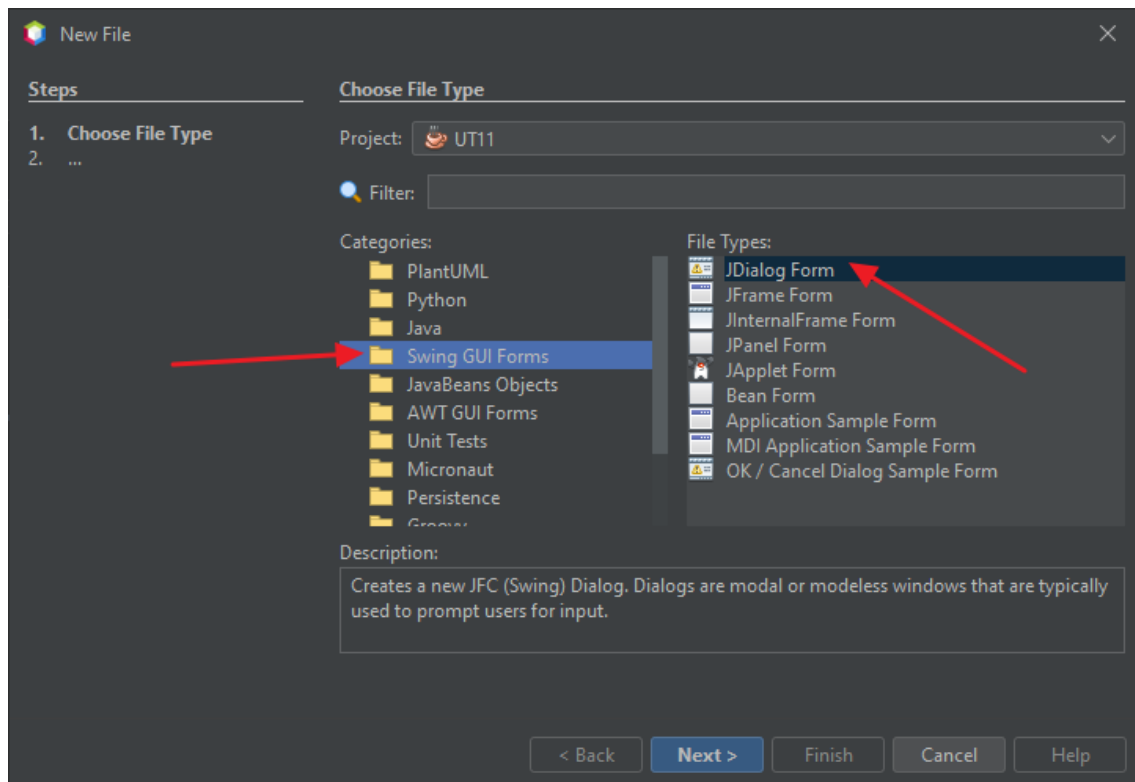


Los diálogos son ventanas emergentes que nos servirán mostrar, recoger o confirmar información, normalmente relacionada con una ventana principal.

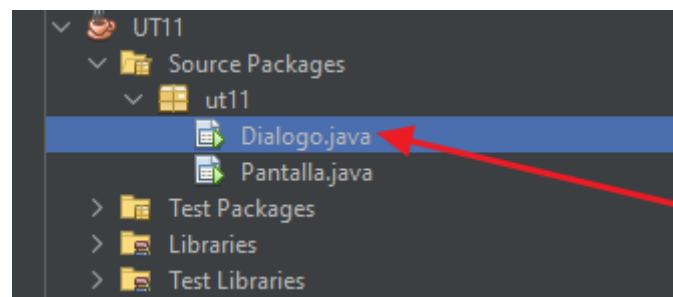
Para crear una ventana de diálogo, dentro de paquete del proyecto vamos a pulsar botón derecho del ratón, indicamos que queremos crear un nuevo elemento y seleccionamos "Other".



De ahí iremos una ventana emergente donde tenemos que seleccionar en la parte izquierda la opción de Swing GUI Form y en la parte izquierda JDialog Form.



Le daremos nombre a la pantalla de diálogo y nos aparecerá en nuestro proyecto:



Es recomendable, eliminar el método main del diálogo, una vez creado, siempre y cuando no vayamos a arrancar la aplicación desde el mismo.

Para que aparezca este diálogo cuando estamos ejecutando, tendremos que crear una instancia del mismo. En el constructor le pasaremos su padre, es decir, la pantalla que le llama y también podremos indicar si queremos que sea modal o no mediante un booleano. Una ventana modal es aquella que cuando aparece no deja acceder a las otras de la aplicación.

```
Dialogo dialogo = new Dialogo(this, true);  
dialogo.setVisible(true);
```

Lo que nosotros vamos a necesitar habitualmente es que se trate de una ventana modal, por tanto, pondremos el segundo valor a true.

Por medio del método setVisible(true) haremos que aparezca en pantalla.

Para cerrar la ventana modal utilizaremos el método dispose(), ya sea desde si misma, o desde la ventana principal, por ejemplo:

```
this.dispose() // si lo hacemos desde el propio diálogo
```

```
dialogo.dispose() // si lo hacemos desde la ventana principal
```

6.7.1. Intercambio de información con diálogos

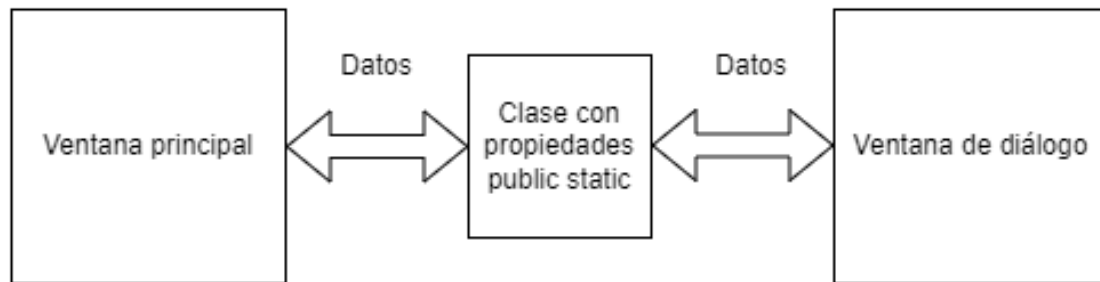
Existen dos formas de intercambiar información entre dos ventanas, ya sean formularios o diálogos. Una es bidireccional y la otra es unidireccional.

6.7.2.1. Intercambio bidireccional

Para poder realizar un intercambio bidireccional de información, por ejemplo, entre una ventana y un diálogo, podemos crearnos una clase que tenga atributos públicos y estáticos.

Será en los atributos de esta clase donde guardaremos la información, sin necesidad de instanciarla, y podremos acceder a ellos desde cualquier parte del programa.

De esta forma, cuando uno de los dos, la ventana principal o el diálogo, realice una modificación en los datos de la clase, ambos tendrán acceso a esos datos.



6.7.2.2. Intercambio unidireccional

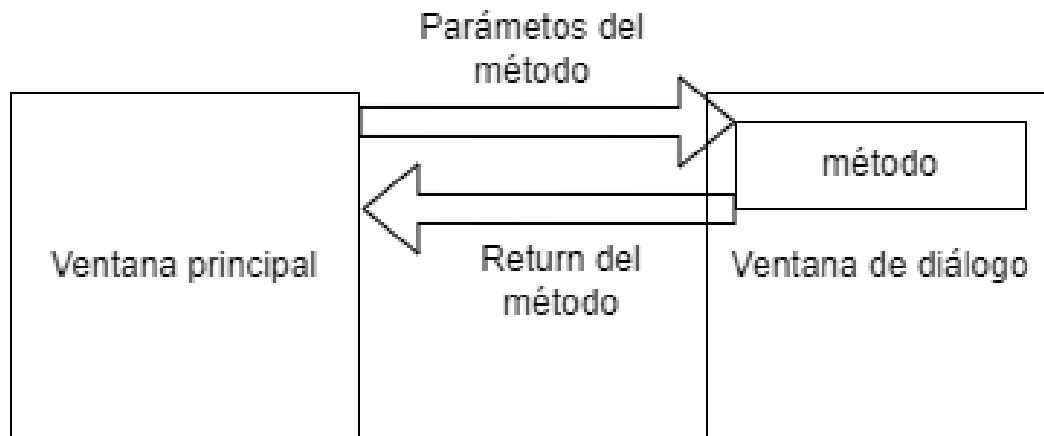
En el caso de solo necesitar pasar información de la ventana principal al diálogo, modificamos el constructor del diálogo y le añadimos los parámetros que necesitemos para pasarle los datos, ya sean con tipos de datos primitivos, o alguna instancia de una clase con la información.

Si lo que necesitamos es pasar la información del diálogo a la ventana principal, lo que haremos será crear un método en el diálogo, que devuelva los datos que necesitemos recibir, ya sea primitivo o una instancia de una clase. Ese método será el encargado hacer que el diálogo se haga visible, y también tendrá el return que devolverá los datos.

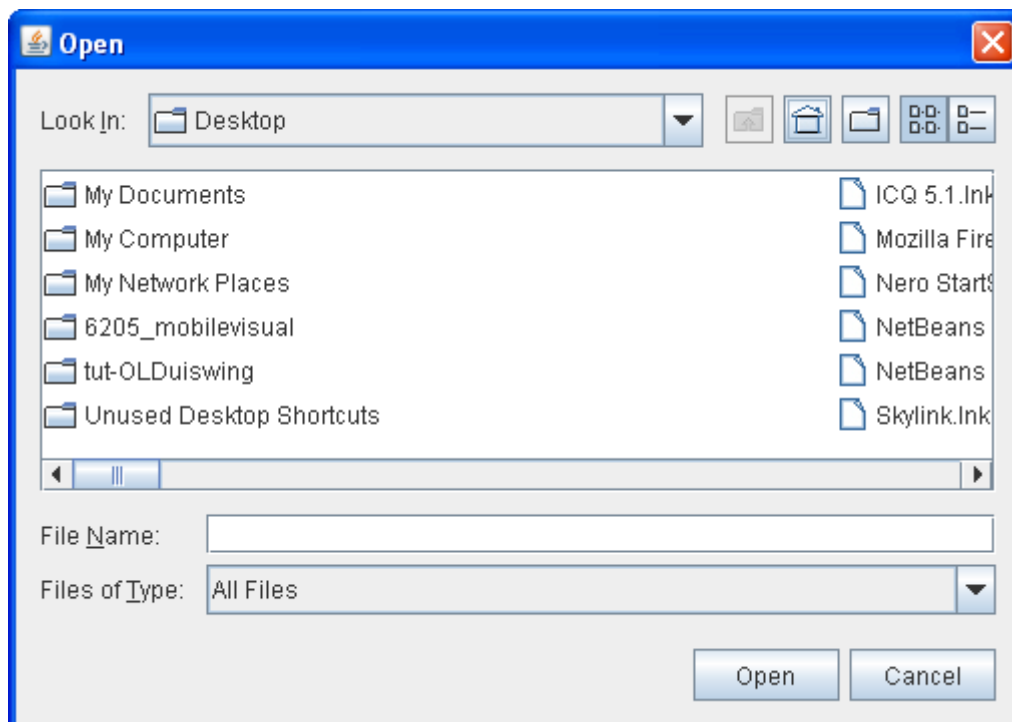
Así, cuando llamemos al método desde la ventana principal, se abrirá el diálogo, y la ejecución del método se quedará pausado hasta que se cierre el diálogo, puesto que le hemos indicado que sea de tipo modal.

Cuando se cierre el diálogo se devolverá el valor que necesitemos, puesto que continuará la ejecución del método llamado desde la ventana principal.

A este segundo caso, también podríamos hacerlo bidireccional pasándole parámetros al método. Pero en este caso el intercambio de información no sería simultáneo como ocurre con el modo que hemos visto anteriormente, sino que primero la ventana principal pasa unos datos al llamar al método y después el diálogo devuelve otros al cerrarse.



6.8. File Chooser



Los file chooser con un tipo especial de diálogo modal, que nos sirve para seleccionar archivos o directorios.

Para hacer que aparezca solo tendremos que crearnos una instancia de la clase `JFileChooser` y mostrarlo. Tenemos dos métodos distintos para mostrarlo que utilizaremos según lo que queramos hacer con el fichero o directorio en cuestión.

Si lo que vamos a hacer es leer el fichero, utilizaremos el método `showOpenDialog()` y le pasaremos como parámetro la ventana que le llama. Este método nos devolverá un valor como resultado que corresponde a una constante. Este método solo nos permitirá seleccionar fichero o directorios ya existentes.

```
public int showOpenDialog(Component parent) throws HeadlessException
```

Los posibles valores constantes que nos podrá devolver son:

```
JFileChooser.CANCEL_OPTION //Si le canceló la operación  
JFileChooser.APPROVE_OPTION //Si se aceptó la operación  
JFileChooser.ERROR_OPTION //Si ocurrió un error durante la operación
```

Este resultado lo tendremos que comprobar cuando se cierre el diálogo, ya que será el momento en el que nos devuelva el resultado.

Si lo que vamos a hacer es escribir en el fichero, utilizaremos el método `showSaveDialog()` que funciona igual que el anterior, pero este nos permite poner el nombre de un fichero que no existe, para que podamos crearlo.

```
public int showSaveDialog(Component parent) throws HeadlessException
```

```
private void btnAbrirArchivosMousePressed(java.awt.event.MouseEvent evt)  
{  
  
    JFileChooser fc = new JFileChooser();  
  
    int seleccion = fc.showOpenDialog(this);  
  
    if(seleccion == JFileChooser.APPROVE_OPTION){  
        File fichero = fc.getSelectedFile( );  
        ...  
    }  
}
```

Al llamar a cualquiera de estos dos métodos, aparecerá el diálogo modal para que seleccionemos un fichero o carpeta y aceptemos, o cancelemos. Hasta ese momento, la ventana principal se quedará en suspenso.

Antes de utilizar estos dos métodos, podemos configurar varias características del diálogo modal. Por ejemplo:

Podemos indicar en qué directorio queremos que comience a explorar:

```
File actual = new File(".");  
fc.setCurrentDirectory(actual);
```

Podemos indicar que solo muestre los ficheros una extensión determinada:

```
FileNameExtensionFilter filtro = new FileNameExtensionFilter("*.CSV",  
"csv");  
fc.setFileFilter(filtro);
```

Podemos indicar que solo se puedan seleccionar ficheros, directorios o ambos:

```
fc.setFileSelectionMode(JFileChooser.FILES_ONLY);  
fc.setFileSelectionMode(JFileChooser.DIRECTORIES_ONLY);  
fc.setFileSelectionMode(JFileChooser.FILES_AND_DIRECTORIES);
```

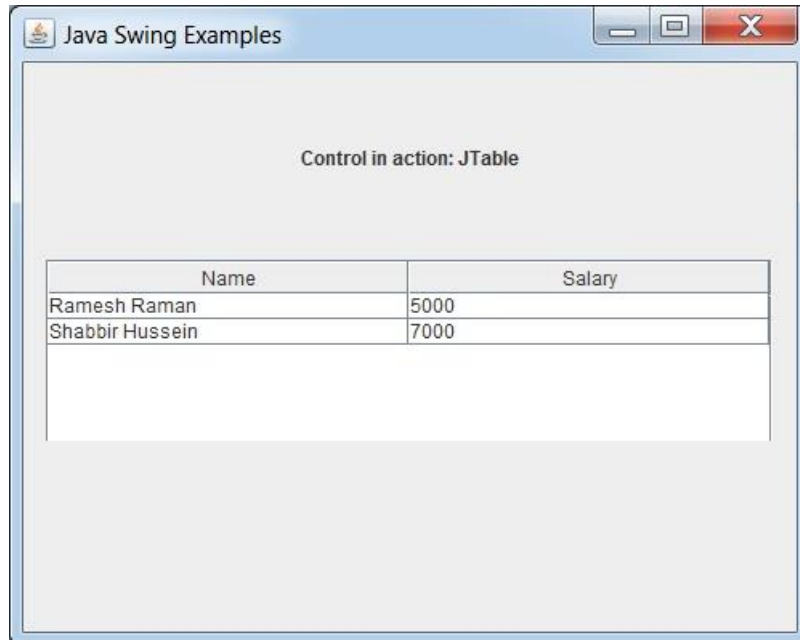
Podemos indicar que se puedan seleccionar varios ficheros:

```
fc.setMultiSelectionEnabled(true);
```

En este caso, debemos utilizar `getSelectedFiles()` para recoger los distintos ficheros seleccionados:

```
File[] ficheros = fc.getSelectedFiles();
```

6.9. Tablas

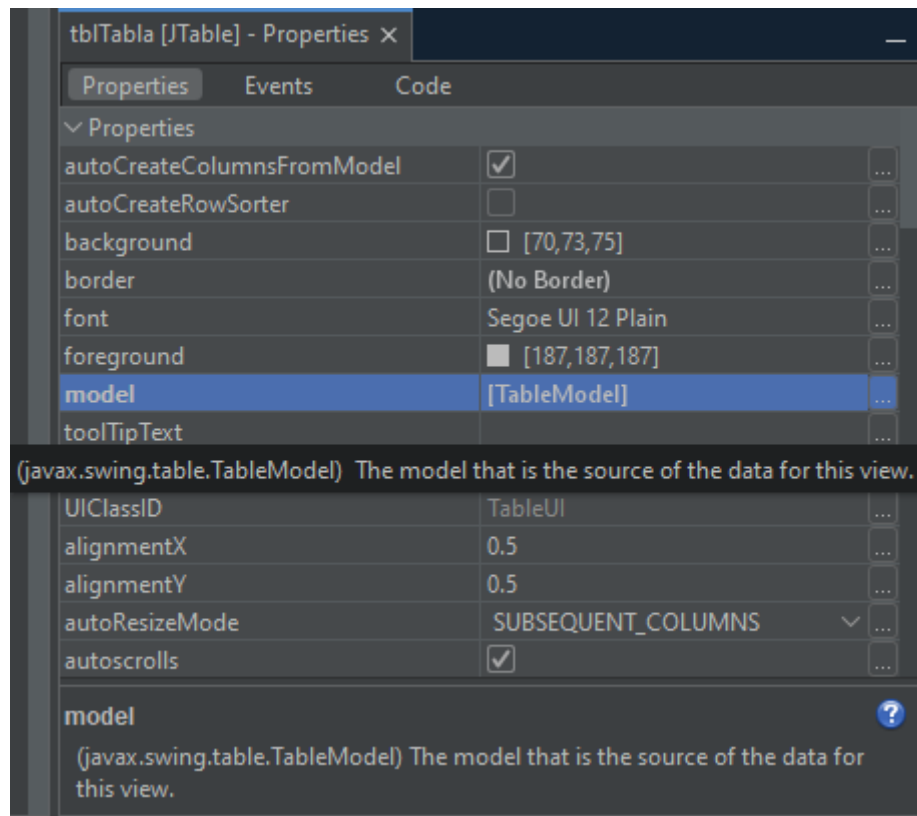


El último componente que vamos a aprender a usar serán las tablas.

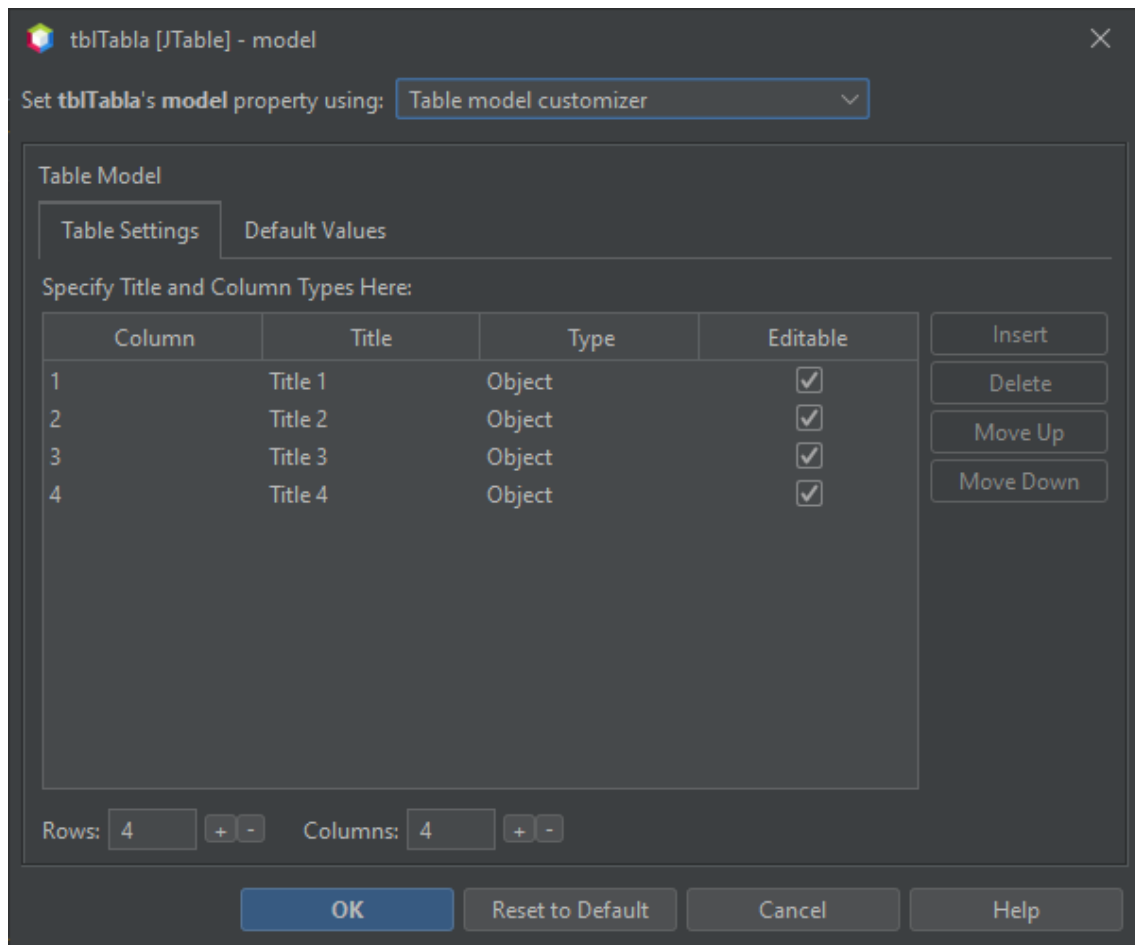
Gracias estos componentes podremos mostrar una gran cantidad información que podamos recopilar de datos proporcionado por el usuario, ficheros o, incluso, bases de datos, como haremos en la siguiente unidad.

Para utilizar una tabla, solo tendremos que arrastrarla desde la paleta a la ventana. Una vez colocada ya nos aparecerán por defecto los nombres de varias columnas. Esto podremos personalizarlo desde las propiedades o por código.

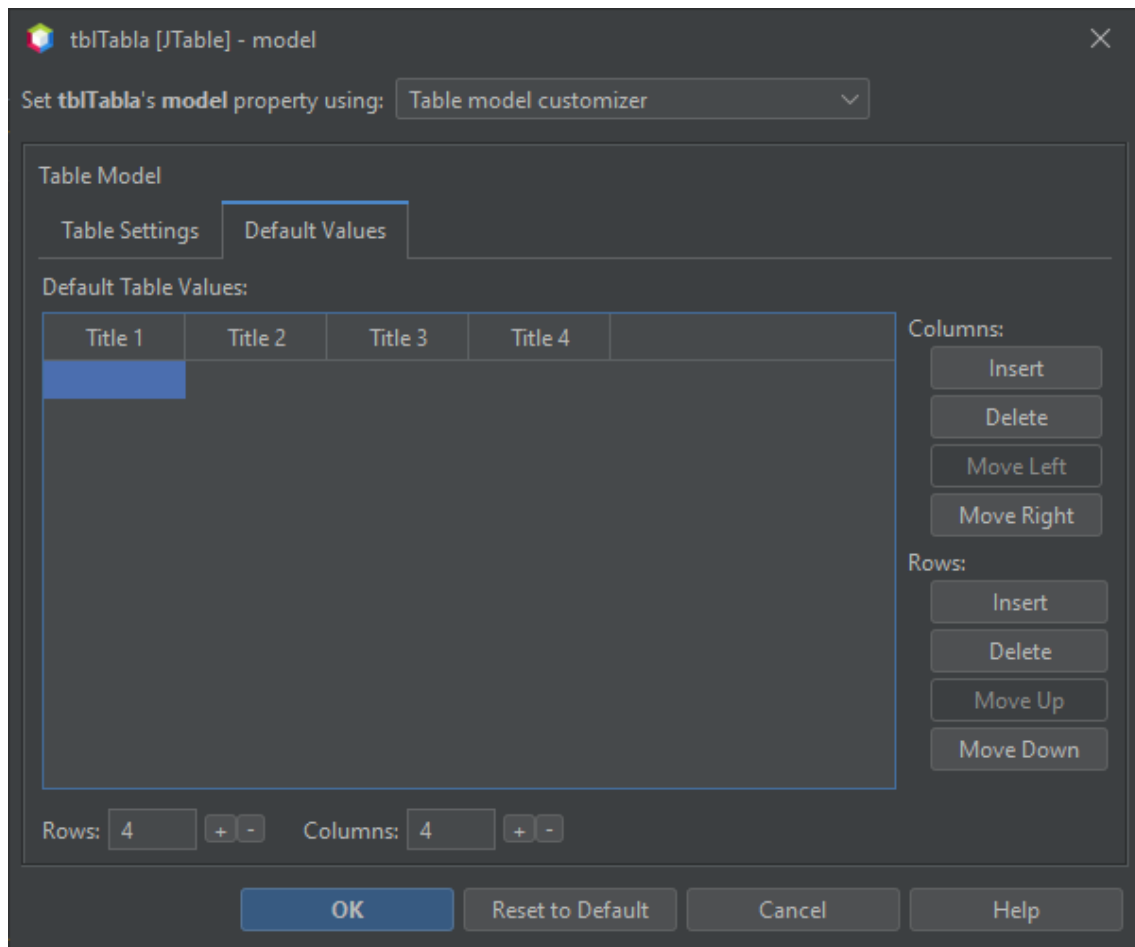
Mediante las propiedades tendremos que acceder a la propiedad "model".



Una vez dentro, en la pestaña Table Settings, nos aparecerán los nombres y características de las columnas (título, tipo de dato, si queremos que se pueda editar, ...)



Y en la pestaña Default Values podremos dar los valores que aparecerán al arrancar la aplicación.



Con respecto a la forma de personalizar las tablas por código vamos a hacer uso de la clase `DefaultTableModel`.

```
DefaultTableModel modelo = new DefaultTableModel();
```

Una vez creado el modelo, le definiremos el número de columnas que vamos a querer con el método `setColumnCount(int numColumnas)` y después le daremos los nombres de las cabeceras. Para esto crearemos un array de Strings, donde cada posición del array sea el nombre de una de las columnas. Finalmente utilizaremos el método `setColumnIdentifiers(Object[] nombCabeceras)` para asignar al modelo los nombres de las cabeceras.

Una vez que ya tenemos las cabeceras, nos centraremos en los datos, los cuales también pasaremos por medio de arrays del tipo `Object`, uno por fila. Cada una de las filas las vamos a añadir al modelo con el método `addRow(Object[] datos)`.

Para terminar, como hacíamos con la lista, vincularemos el modelo a la tabla con el método `setModel(DefaultTableModel modelo)`.

Veamos un ejemplo.

```
DefaultTableModel modelo = new DefaultTableModel();

modelo.setColumnCount(4);
String[] cabeceras = {"Id", "Nombre", "Apellido", "Telefono"};
modelo.setColumnIdentifiers(cabeceras);

Object[] fila1 = {1, "Pepe", "Torres", 656789034};
Object[] fila2 = {2, "Sara", "Pera", 676889034};
Object[] fila3 = {3, "Ana", "Limón", 600884534};

modelo.addRow(fila1);
modelo.addRow(fila2);
modelo.addRow(fila3);

tblTabla.setModel(modelo);
```

Id	Nombre	Apellido	Telefono
1	Pepe	Torres	656789034
2	Sara	Pera	676889034
3	Ana	Limón	600884534

Si deseamos que en nuestra tabla se puedan seleccionar las filas, pero no se puedan editar las celdas debemos añadir el siguiente código:

```
tblTabla.setDefaultEditor(Object.class, null);
```

De lo contrario nuestra tabla nos permitirá editar los datos que contiene.

Otros métodos que nos pueden ser útiles son:

- `getSelectedColumn()`: devuelve el índice la columna seleccionada. Devolverá -1 no si no hay ninguna seleccionada.
- `getSelectedColumnCount()`: devuelve el número de columnas seleccionadas.

- `getSelectedColumns()`: devuelve un array con los índices de las columnas seleccionadas. Devolverá un array vacío si no hay ninguna columna seleccionada.
- `getSelectedRow()`: devuelve el índice la fila seleccionada. Devolverá -1 si no hay ninguna seleccionada.
- `getSelectedRowCount()`: devuelve el número de filas seleccionadas.
- `getSelectedRows()`: devuelve un array con los índices de las filas seleccionadas. Devolverá un array vacío si no hay ninguna fila seleccionada.
- `getRowCount()`: devuelve el número de filas de la tabla.
- `getValueAt(int fila, int columna)`: devolverá el dato que hay en la posición indicada.

Por último, si lo que necesitamos es eliminar una fila de la tabla, vamos a tener que capturar su modelo y eliminar la fila mediante el método `removeRow(int index)` del mismo.

```
DefaultTableModel mod = (DefaultTableModel) tblTabla.getModel();  
mod.removeRow(1);
```

Id	Nombre	Apellido	Telefono
1	Pepe	Torres	656789034
3	Ana	Limón	600884534